
Comprehensive Code Documentation

CSU2220 - Generative AI for Building Applications

AI Implementation Analysis

December 19, 2025

Contents

1	Original Code: code1.py	8
1.1	Explanation of the Code	8
1.2	Flowchart	10
2	Ollama Equivalent Code	11
2.1	Diff Explanation	11
2.2	Explanation of the Diff	12
3	Hugging Face Equivalent Code	13
3.1	Diff Explanation	13
3.2	Explanation of the Diff	13
4	Original Code: code2.py	15
4.1	Explanation of the Code	15
4.2	Flowchart	16
5	Ollama Equivalent Code	17
5.1	Diff Explanation	17
5.2	Explanation of the Diff	18
6	Hugging Face Equivalent Code	18
6.1	Diff Explanation	18
6.2	Explanation of the Diff	19
7	Original Code: code3.py	20
7.1	Explanation of the Code	20

7.2 Flowchart	21
8 Ollama Equivalent Code	21
8.1 Diff Explanation	22
8.2 Explanation of the Diff	22
9 Hugging Face Equivalent Code	22
9.1 Diff Explanation	23
9.2 Explanation of the Diff	23
10 Original Code: code4.py	24
10.1 Explanation of the Code	24
10.2 Flowchart	25
11 Ollama Equivalent Code	25
11.1 Diff Explanation	26
11.2 Explanation of the Diff	26
12 Hugging Face Equivalent Code	26
12.1 Diff Explanation	27
12.2 Explanation of the Diff	27
13 Original Code: code5.py	28
13.1 Explanation of the Code	28
13.2 Flowchart	29
14 Ollama Equivalent Code	30
14.1 Diff Explanation	30
14.2 Explanation of the Diff	31
15 Hugging Face Equivalent Code	31
15.1 Diff Explanation	31
15.2 Explanation of the Diff	32
16 Original Code: code6.py	33
16.1 Explanation of the Code	33
16.2 Flowchart	34
17 Ollama Equivalent Code	34
17.1 Diff Explanation	35
17.2 Explanation of the Diff	35
18 Hugging Face Equivalent Code	35
18.1 Diff Explanation	36
18.2 Explanation of the Diff	36
19 Original Code: code7.py	37
19.1 Explanation of the Code	37
19.2 Flowchart	39
20 Ollama Equivalent Code	39
20.1 Diff Explanation	40
20.2 Explanation of the Diff	41
21 Hugging Face Equivalent Code	41
21.1 Diff Explanation	41
21.2 Explanation of the Diff	42
22 Original Code: code8.py	43

22.1 Explanation of the Code	44
22.2 Flowchart	45
23 Ollama Equivalent Code	46
23.1 Diff Explanation	47
23.2 Explanation of the Diff	47
24 Hugging Face Equivalent Code	47
24.1 Diff Explanation	48
24.2 Explanation of the Diff	49
25 Original Code: code9.py	50
25.1 Explanation of the Code	51
25.2 Flowchart	52
26 Ollama Equivalent Code	53
26.1 Diff Explanation	54
26.2 Explanation of the Diff	54
27 Hugging Face Equivalent Code	55
27.1 Diff Explanation	55
27.2 Explanation of the Diff	56
28 Original Code: code9A.py	57
28.1 Explanation of the Code	58
28.2 Flowchart	59
29 Ollama Equivalent Code	60
29.1 Diff Explanation	61
29.2 Explanation of the Diff	61
30 Hugging Face Equivalent Code	61
30.1 Diff Explanation	62
30.2 Explanation of the Diff	63
31 Original Code: code9B.py	64
31.1 Explanation of the Code	65
31.2 Flowchart	66
32 Ollama Equivalent Code	67
32.1 Diff Explanation	68
32.2 Explanation of the Diff	68
33 Hugging Face Equivalent Code	68
33.1 Diff Explanation	69
33.2 Explanation of the Diff	70
34 Original Code: code9C.py	71
34.1 Explanation of the Code	72
34.2 Flowchart	73
35 Ollama Equivalent Code	74
35.1 Diff Explanation	75
35.2 Explanation of the Diff	75
36 Hugging Face Equivalent Code	76
36.1 Diff Explanation	76
36.2 Explanation of the Diff	77

37 Original Code: code10.py	78
37.1 Explanation of the Code	78
37.2 Flowchart	80
38 Ollama Equivalent Code	80
38.1 Diff Explanation	81
38.2 Explanation of the Diff	81
39 Hugging Face Equivalent Code	82
39.1 Diff Explanation	82
39.2 Explanation of the Diff	83
40 Original Code: code11.py	84
40.1 Explanation of the Code	84
40.2 Flowchart	86
41 Ollama Equivalent Code	87
41.1 Diff Explanation	87
41.2 Explanation of the Diff	88
42 Hugging Face Equivalent Code	88
42.1 Diff Explanation	88
42.2 Explanation of the Diff	89
43 Original Code: code12.py	90
43.1 Explanation of the Code	92
43.2 Flowchart	93
44 Ollama Equivalent Code	94
44.1 Diff Explanation	96
44.2 Explanation of the Diff	96
45 Hugging Face Equivalent Code	97
45.1 Diff Explanation	99
45.2 Explanation of the Diff	99
46 Original Code: code13.py	100
46.1 Explanation of the Code	100
46.2 Flowchart	101
47 Ollama Equivalent Code	101
47.1 Diff Explanation	102
47.2 Explanation of the Diff	103
48 Hugging Face Equivalent Code	103
48.1 Diff Explanation	103
49 Original Code: code13A.py	104
49.1 Explanation of the Code	105
49.2 Flowchart	106
50 Ollama Equivalent Code	106
50.1 Diff Explanation	107
50.2 Explanation of the Diff	108
51 Hugging Face Equivalent Code	108
51.1 Diff Explanation	108
52 Original Code: code14.py	109

52.1 Explanation of the Code	109
52.2 Flowchart	110
53 Ollama Equivalent Code	110
53.1 Diff Explanation	111
53.2 Explanation of the Diff	111
54 Hugging Face Equivalent Code	111
54.1 Diff Explanation	112
54.2 Explanation of the Diff	112
55 Original Code: code14A.py	113
55.1 Explanation of the Code	113
55.2 Flowchart	114
56 Ollama Equivalent Code	114
56.1 Diff Explanation	114
56.2 Explanation of the Diff	114
57 Hugging Face Equivalent Code	115
57.1 Diff Explanation	115
58 Original Code (src/code15.py)	116
59 Explanation	116
59.1 What	117
59.2 Why	117
59.3 How	117
60 Flowchart	118
61 Original Code (src/code15A.py)	119
62 Explanation	120
62.1 What	120
62.2 Why	121
62.3 How	121
63 Flowchart	122
64 Ollama Equivalent Code	123
64.1 Diff Explanation	124
65 Hugging Face Equivalent Code	124
65.1 Diff Explanation	126
66 Original Code (src/code16.py)	127
67 Explanation	128
67.1 What	128
67.2 Why	128
67.3 How	128
68 Flowchart	130
69 Ollama Equivalent Code	131
69.1 Diff Explanation	132
70 Hugging Face Equivalent Code	132
70.1 Diff Explanation	133
71 Original Code (src/code17.py)	135
72 Explanation	138

72.1 What	138
72.2 Why	138
72.3 How	138
73 Flowchart	139
74 Ollama Equivalent Code	139
74.1 Diff Explanation	142
75 Hugging Face Equivalent Code	143
75.1 Diff Explanation	145

1 Original Code: code1.py

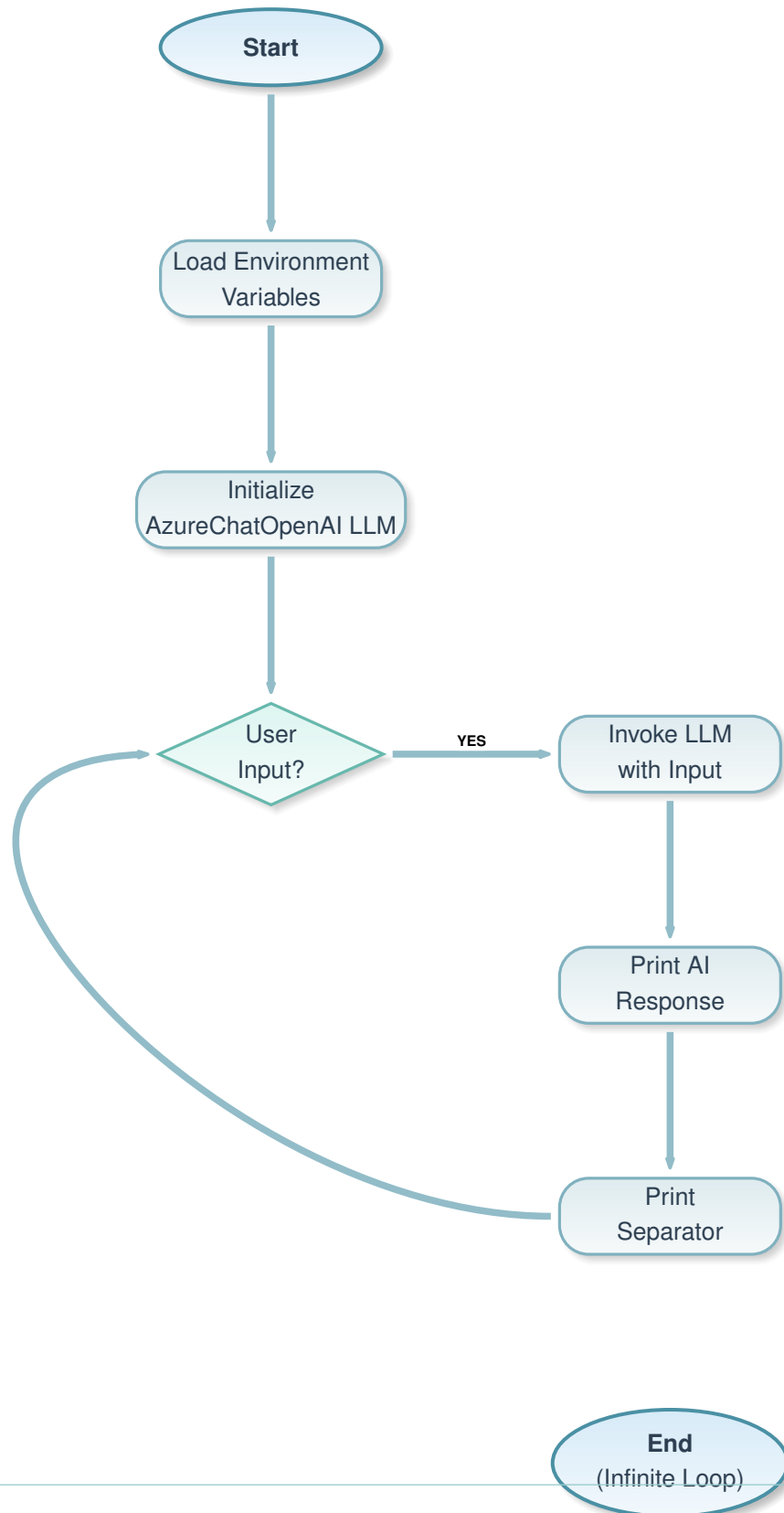
```
1 from langchain.chat_models import AzureChatOpenAI
2 import os
3 from dotenv import load_dotenv
4
5
6 load_dotenv()
7
8 llm = AzureChatOpenAI(
9     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
10    openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
11    openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
12    deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
13    model_name="gpt-4o",
14    temperature=0.7,
15 )
16
17 while True:
18     input_text=input("Enter your query: ")
19     result = llm.invoke(input_text)
20     print("AI:",result.content)
21     print("-----")
```

1.1 Explanation of the Code

- **What:** This code implements a simple interactive chatbot that continuously takes user queries as input and generates responses using an Azure OpenAI model via LangChain. It prints the AI's response and separates interactions with a line.
- **Why:** The code demonstrates basic integration with cloud-based AI models like Azure OpenAI for conversational AI. It uses LangChain for abstraction, dotenv for secure environment variable management, and a loop for continuous interaction. This approach is chosen for simplicity in prototyping AI chatbots, ensuring API keys are not hardcoded, and leveraging LangChain's unified interface for different LLMs.
- **How:**
 - Load environment variables from a .env file using dotenv to securely access API credentials.

- Initialize an `AzureChatOpenAI` instance with the loaded credentials, specifying the model (`gpt-4o`) and temperature (`0.7` for creativity).
- Enter an infinite loop: Prompt the user for input, invoke the LLM with the input to generate a response, print the AI's content from the result object, and print a separator line.

1.2 Flowchart



2 Ollama Equivalent Code

```
1 from langchain_ollama import OllamaLLM
2
3 llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
4
5 while True:
6     input_text = input("Enter your query: ")
7     result = llm.invoke(input_text)
8     print("AI:", result)
9     print("-----")
```

2.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- llm = AzureChatOpenAI(
-     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
-     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
-     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
-     openai_api_version=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
-     model_name="gpt-4o",
-     temperature=0.7,
- )
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

Modified Code

```
print("AI:", result.content) -> print("AI:", result)
```

2.2 Explanation of the Diff

- **Removed imports and initialization:** The Azure-specific imports and environment variable loading are removed because Ollama runs locally and doesn't require API keys or cloud configuration.
- **Changed LLM initialization:** Switched to OllamaLLM for local model execution, using a specific model name "qwen3:4b" instead of Azure's gpt-4o.
- **Modified print statement:** In Ollama, the result is directly the string, not an object with .content, so no need to access .content.
- **Why:** These changes adapt the code from cloud-based Azure OpenAI to local Ollama deployment, simplifying setup by removing authentication and using local model inference.

3 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 while True:
6     input_text = input("Enter your query: ")
7     result = pipe(input_text, max_length=100, temperature=0.7,
8     ↪ num_return_sequences=1)
9     print("AI:", result[0]['generated_text'])
10    print("-----")
```

3.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- llm = AzureChatOpenAI(...)
- result = llm.invoke(input_text)
- print("AI:", result.content)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result = pipe(input_text, max_length=100, temperature=0.7,
+               num_return_sequences=1)
+ print("AI:", result[0]['generated_text'])
```

3.2 Explanation of the Diff

- **Removed LangChain and Azure setup:** Replaced with Hugging Face transformers for direct model interaction.
- **Changed to pipeline:** Uses text-generation pipeline with a specific model, adding parameters like `max_length` and `num_return_sequences` for generation control.

- **Modified result handling:** The result is a list of dictionaries, accessing the generated text accordingly.
- **Why:** Hugging Face provides a different API focused on transformers models, requiring different initialization and invocation methods, and handles generation parameters explicitly.

4 Original Code: code2.py

```
1  import os
2  from dotenv import load_dotenv
3  from langchain_openai import AzureChatOpenAI
4
5  # Load environment variables
6  load_dotenv()
7
8  # Fetch from .env or directly assign as needed
9  api_key = ""
10 api_base = "https://chatgpt-key.openai.azure.com/"
11 deployment_name = "gpt-4o-2"
12 api_version = "2024-12-01-preview" # Ensure this is a valid API version
13
14 print(f"API Key: {api_key}")
15
16 # Initialize the model
17 llm = AzureChatOpenAI(
18     deployment_name=deployment_name,
19     api_version=api_version,
20     api_key=api_key,
21     azure_endpoint=api_base,
22     model_name="gpt-4o",
23     temperature=0.7,
24     max_tokens=100, # Maximum number of tokens in the response
25 )
26
27
28 # Use the predict method instead of chat
29 response = llm.predict("Hello, how are you?")
30
31 # Print the response
32 print(response)
```

4.1 Explanation of the Code

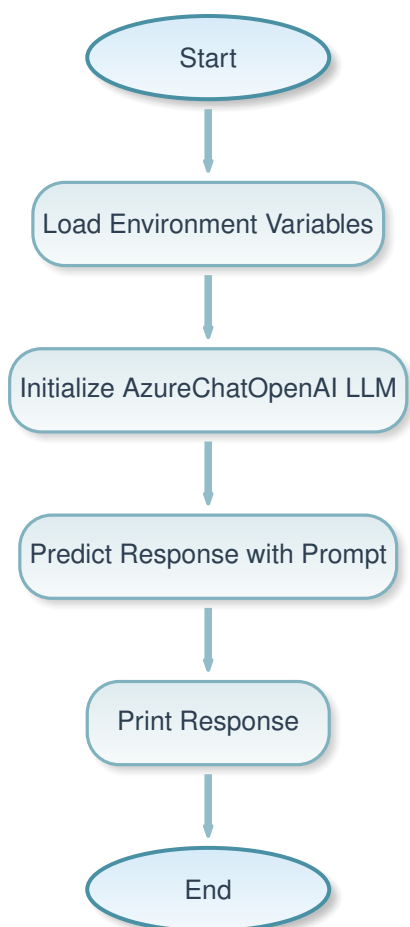
- **What:** This code initializes an Azure OpenAI model using LangChain and generates a single response to the prompt "Hello, how are you?" using the predict method, then prints the response.
- **Why:** It demonstrates basic setup and single-query interaction with Azure OpenAI via

LangChain. The use of predict method shows an alternative to invoke for simple text generation. Environment variables are loaded for security, though here API key is empty (placeholder). This approach is for testing or simple API calls without continuous interaction.

- **How:**

- Load environment variables (though API key is hardcoded empty here).
- Print the API key (for debugging).
- Initialize AzureChatOpenAI with deployment details, API version, key, endpoint, model name, temperature, and max tokens.
- Call predict with a fixed prompt to get a response.
- Print the response string.

4.2 Flowchart



5 Ollama Equivalent Code

```
1 from langchain_ollama import OllamaLLM
2
3 llm = OllamaLLM(
4     model="qwen3:4b",
5     temperature=0.7,
6     max_tokens=100,
7 )
8
9 response = llm.invoke("Hello, how are you?")
10
11 print(response)
```

5.1 Diff Explanation

— Removed Code

```
- import os
- from dotenv import load_dotenv
- from langchain_openai import AzureChatOpenAI
- load_dotenv()
- api_key = ""
- api_base = "https://chatgpt-key.openai.azure.com/"
- deployment_name = "gpt-4o-2"
- api_version = "2024-12-01-preview"
- print(f"API Key: {api_key}")
- llm = AzureChatOpenAI(...)
- response = llm.predict("Hello, how are you?")
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7, max_tokens=100)
```

Modified Code

```
response = llm.invoke("Hello, how are you?")
```

5.2 Explanation of the Diff

- **Removed Azure setup:** Eliminated environment loading, API key handling, and Azure-specific parameters since Ollama is local.
- **Changed initialization:** Used OllamaLLM with model name and parameters.
- **Modified invocation:** Switched from predict to invoke, as Ollama uses invoke for generation.
- **Why:** Adapts to Ollama's local execution, removing cloud dependencies and using LangChain's Ollama wrapper for consistency.

6 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 response = pipe("Hello, how are you?", max_new_tokens=50, temperature=0.7,
6                 ↪ num_return_sequences=1)
7
8 print(response[0]['generated_text'])
```

6.1 Diff Explanation

— Removed Code

```
- import os
- from dotenv import load_dotenv
- from langchain_openai import AzureChatOpenAI
- load_dotenv()
- api_key = ""
- ... (Azure init)
- response = llm.predict("Hello, how are you?")
- print(response)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ response = pipe("Hello, how are you?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print(response[0]['generated_text'])
```

6.2 Explanation of the Diff

- **Removed LangChain and Azure:** Replaced with direct Hugging Face pipeline.
- **Changed to pipeline generation:** Used text-generation pipeline with `max_new_tokens` instead of `max_tokens`.
- **Modified output handling:** Accessed the generated text from the result list.
- **Why:** Hugging Face transformers provide a different interface for model loading and generation, focusing on local model execution without LangChain abstraction.

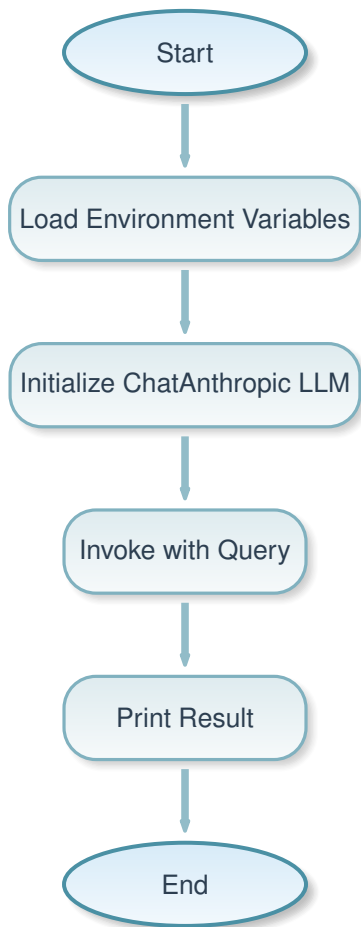
7 Original Code: code3.py

```
1  #ChatAnthropic - cluade
2
3  from langchain_anthropic import ChatAnthropic
4  from dotenv import load_dotenv
5  import os
6  load_dotenv()
7  llm=ChatAnthropic(temperature=0.7, model="claude-2", anthropic_api_key=os.
    ↪   getenv("ANTHROPIC_API_KEY"))
8  result=llm.invoke("What is the capital of France?")
9  print(result)
```

7.1 Explanation of the Code

- **What:** This code initializes a ChatAnthropic model (Claude) using LangChain and generates a response to the query "What is the capital of France?", then prints the result.
- **Why:** Demonstrates integration with Anthropic's Claude model via LangChain for AI-powered question answering. Uses environment variables for API key security. This approach is for simple, single-turn interactions with proprietary models like Claude.
- **How:**
 - Load environment variables from .env file.
 - Initialize ChatAnthropic with temperature, model name, and API key from env.
 - Invoke the model with a fixed question.
 - Print the result object (which contains the response).

7.2 Flowchart



8 Ollama Equivalent Code

```
1 from langchain_ollama import OllamaLLM
2
3 llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
4
5 result = llm.invoke("What is the capital of France?")
6
7 print(result)
```

8.1 Diff Explanation

— Removed Code

```
- from langchain_anthropic import ChatAnthropic
- from dotenv import load_dotenv
- import os
- load_dotenv()
- llm=ChatAnthropic(temperature=0.7, model="claude-2",
anthropic_api_key=os.getenv("ANTHROPIC_API_KEY"))
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

8.2 Explanation of the Diff

- **Removed Anthropic setup:** Eliminated API key loading and ChatAnthropic initialization.
- **Changed to Ollama:** Used OllamaLLM for local execution.
- **Why:** Switches from cloud-based Anthropic API to local Ollama model, removing authentication requirements.

9 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 result = pipe("What is the capital of France?", max_new_tokens=50,
6               ↪ temperature=0.7, num_return_sequences=1)
7
8 print(result[0]['generated_text'])
```

9.1 Diff Explanation

— Removed Code

```
- from langchain_anthropic import ChatAnthropic
- from dotenv import load_dotenv
- import os
- load_dotenv()
- llm=ChatAnthropic(...)
- result=llm.invoke("What is the capital of France?")
- print(result)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result = pipe("What is the capital of France?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print(result[0]['generated_text'])
```

9.2 Explanation of the Diff

- **Removed LangChain Anthropic:** Replaced with Hugging Face pipeline.
- **Changed invocation and output:** Used pipeline generate method and accessed text from result.
- **Why:** Adapts to Hugging Face's direct model interface for text generation.

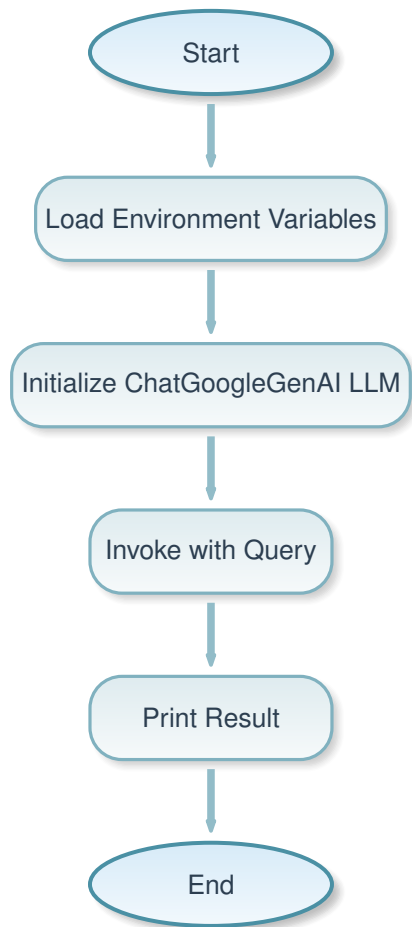
10 Original Code: code4.py

```
1 from langchain_google_genai import ChatGoogleGenAI
2 from dotenv import load_dotenv
3 import os
4 load_dotenv()
5
6 llm=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
7     ↪ google_api_key=os.getenv("GOOGLE_API_KEY"))
8
9 result=llm.invoke("What is the capital of France?")
10 print(result)
```

10.1 Explanation of the Code

- **What:** This code initializes a ChatGoogleGenAI model (Gemini) using LangChain and generates a response to "What is the capital of France?", then prints it.
- **Why:** Shows integration with Google's Gemini model via LangChain for AI responses. Uses env vars for API key. Suitable for single queries to Google's AI.
- **How:**
 - Load env vars.
 - Init ChatGoogleGenAI with temp, model, API key.
 - Invoke with query.
 - Print result.

10.2 Flowchart



11 Ollama Equivalent Code

```
1 from langchain_ollama import OllamaLLM
2
3 llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
4
5 result = llm.invoke("What is the capital of France?")
6
7 print(result)
```

11.1 Diff Explanation

— Removed Code

```
- from langchain_google_genai import ChatGoogleGenAI
- from dotenv import load_dotenv
- import os
- load_dotenv()
- llm=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
  google_api_key=os.getenv("GOOGLE_API_KEY"))
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
```

11.2 Explanation of the Diff

- **Removed Google setup:** Eliminated API key and ChatGoogleGenAI.
- **Changed to Ollama:** Local model execution.
- **Why:** From cloud Google API to local Ollama.

12 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 result = pipe("What is the capital of France?", max_new_tokens=50,
6               ↪ temperature=0.7, num_return_sequences=1)
7
8 print(result[0]['generated_text'])
```

12.1 Diff Explanation

— Removed Code

```
- from langchain_google_genai import ChatGoogleGenAI
- ...
- result=llm.invoke("What is the capital of France?")
- print(result)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result = pipe("What is the capital of France?", max_new_tokens=50, temperature=0.7,
+ num_return_sequences=1)
+ print(result[0]['generated_text'])
```

12.2 Explanation of the Diff

- **Removed LangChain Google:** To Hugging Face pipeline.
- **Changed invocation:** Pipeline generate.
- **Why:** Direct transformers usage.

13 Original Code: code5.py

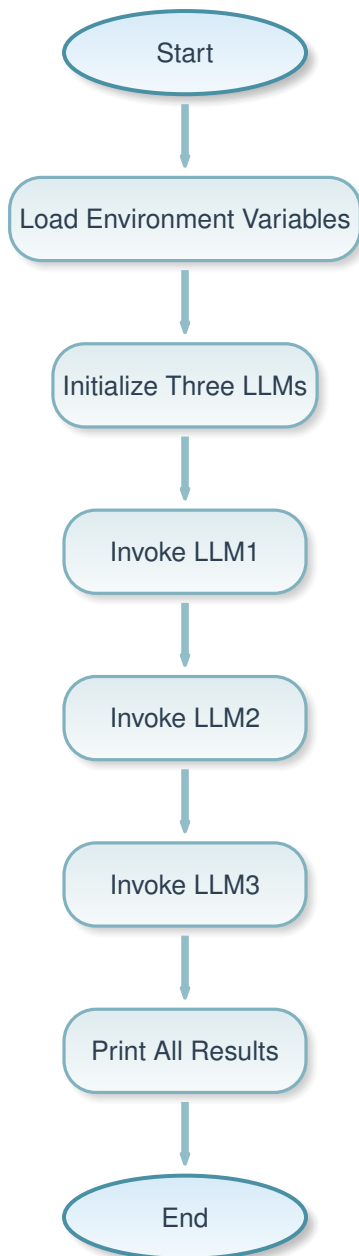
```
1  from langchain_openai import ChatOpenAI
2
3  from langchain_google_genai import ChatGoogleGenAI
4
5  from langchain_anthropic import ChatAnthropic
6
7  from dotenv import load_dotenv
8  import os
9
10
11 llm1=ChatOpenAI(model="gpt-3.5-turbo",temperature=0.5,openai_api_key=os.
    ↪ getenv("OPENAI_API_KEY"))
12 llm2=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
    ↪ google_api_key=os.getenv("GOOGLE_API_KEY"))
13 llm3=ChatAnthropic(temperature=0.7, model="claude-2", anthropic_api_key=os
    ↪ .getenv("ANTHROPIC_API_KEY"))
14
15 result1=llm1.invoke("Opinion on Pakistan?")
16 result2=llm2.invoke("Opinion on Pakistan?")
17 result3=llm3.invoke("Opinion on Pakistan?")
18
19 print("OpenAI result: ",result1)
20 print("Google result: ",result2)
21 print("Anthropic result: ",result3)
```

13.1 Explanation of the Code

- **What:** This code initializes three different LLM models (OpenAI GPT-3.5, Google Gemini, Anthropic Claude) using LangChain and generates responses to the same query "Opinion on Pakistan?" from each, then prints them labeled by provider.
- **Why:** Demonstrates comparing outputs from multiple AI providers for the same prompt, highlighting differences in responses. Uses env vars for API keys. This is useful for evaluating model performance or biases across platforms.
- **How:**
 - Load env vars.
 - Initialize three LLMs with different providers, models, and temps.

- Invoke each with the same query.
- Print results with labels.

13.2 Flowchart



14 Ollama Equivalent Code

```

1  from langchain_ollama import OllamaLLM
2
3  llm1 = OllamaLLM(model="qwen3:4b", temperature=0.5)
4  llm2 = OllamaLLM(model="qwen3:4b", temperature=0.7)  # Change model name
               ↪ to ones installed for better experience
5  llm3 = OllamaLLM(model="qwen3:4b", temperature=0.7)  # Change model name
               ↪ to ones installed for better experience
6
7  result1 = llm1.invoke("Opinion on Pakistan?")
8  result2 = llm2.invoke("Opinion on Pakistan?")
9  result3 = llm3.invoke("Opinion on Pakistan?")
10
11 print("OpenAI result: ", result1)
12 print("Google result: ", result2)
13 print("Anthropic result: ", result3)

```

14.1 Diff Explanation

— Removed Code

```

- from langchain_openai import ChatOpenAI
- from langchain_google_genai import ChatGoogleGenAI
- from langchain_anthropic import ChatAnthropic
- from dotenv import load_dotenv
- import os
- llm1=ChatOpenAI(model="gpt-3.5-turbo",temperature=0.5,openai_api_key=os.getenv("OPENAI_
API_KEY"))
- llm2=ChatGoogleGenAI(temperature=0.7, model="gemini-1.5-pro",
google_api_key=os.getenv("GOOGLE_API_KEY"))
- llm3=ChatAnthropic(temperature=0.7, model="claude-2",
anthropic_api_key=os.getenv("ANTHROPIC_API_KEY"))

```

+++ Added Code

```

+ from langchain_ollama import OllamaLLM
+ llm1 = OllamaLLM(model="qwen3:4b", temperature=0.5)
+ llm2 = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ llm3 = OllamaLLM(model="qwen3:4b", temperature=0.7)

```

14.2 Explanation of the Diff

- **Removed multiple provider setups:** Eliminated API keys and different LangChain imports.
- **Changed to multiple Ollama instances:** Used same model with varying temperatures for comparison.
- **Why:** Adapts to local Ollama, simulating multi-model comparison with different configs instead of different providers.

15 Hugging Face Equivalent Code

```

1  from transformers import pipeline
2
3  pipe1 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4  pipe2 = pipeline("text-generation", model="Qwen/Qwen3-0.6B") # Change
   ↳ model name to ones installed for better experience
5  pipe3 = pipeline("text-generation", model="Qwen/Qwen3-0.6B") # Change
   ↳ model name to ones installed for better experience
6
7  result1 = pipe1("Opinion on Pakistan?", max_new_tokens=50, temperature
   ↳ =0.5, num_return_sequences=1)
8  result2 = pipe2("Opinion on Pakistan?", max_new_tokens=50, temperature
   ↳ =0.7, num_return_sequences=1)
9  result3 = pipe3("Opinion on Pakistan?", max_new_tokens=50, temperature
   ↳ =0.7, num_return_sequences=1)
10
11 print("OpenAI result: ", result1[0]['generated_text'])
12 print("Google result: ", result2[0]['generated_text'])
13 print("Anthropic result: ", result3[0]['generated_text'])

```

15.1 Diff Explanation

— Removed Code

```

- from langchain_openai import ChatOpenAI
- ...
- llm1=...
- llm2=...
- llm3=...

```

```
- result1=llm1.invoke("Opinion on Pakistan?")
- result2=llm2.invoke("Opinion on Pakistan?")
- result3=llm3.invoke("Opinion on Pakistan?")
- print("OpenAI result: ",result1)
- print("Google result: ",result2)
- print("Anthropic result: ",result3)
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe1 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ pipe2 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ pipe3 = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ result1 = pipe1("Opinion on Pakistan?", max_new_tokens=50, temperature=0.5,
num_return_sequences=1)
+ result2 = pipe2("Opinion on Pakistan?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ result3 = pipe3("Opinion on Pakistan?", max_new_tokens=50, temperature=0.7,
num_return_sequences=1)
+ print("OpenAI result: ", result1[0]['generated_text'])
+ print("Google result: ", result2[0]['generated_text'])
+ print("Anthropic result: ", result3[0]['generated_text'])
```

15.2 Explanation of the Diff

- **Removed LangChain multi-provider:** Replaced with multiple Hugging Face pipelines.
- **Changed invocation and output:** Used pipeline generate and accessed text.
- **Why:** Uses direct transformers for local multi-instance comparison.

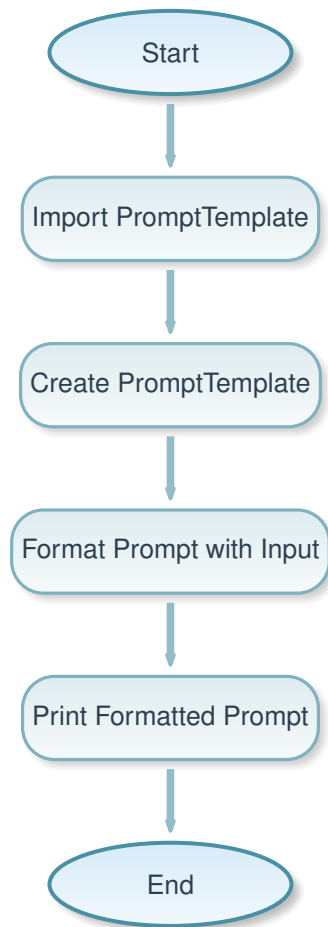
16 Original Code: code6.py

```
1 from langchain.prompts import PromptTemplate
2
3 prompt=PromptTemplate(
4     input_variables=["input"],
5     template="What is the capital of {input}?",
6 )
7
8 print(prompt.format(input="China"))
```

16.1 Explanation of the Code

- **What:** This code defines a PromptTemplate with a placeholder for input, formats it with "China", and prints the resulting prompt string.
- **Why:** Demonstrates LangChain's PromptTemplate for dynamic prompt creation. Useful for reusable prompts with variables. Here, it just formats and prints without invoking an LLM.
- **How:**
 - Import PromptTemplate.
 - Create template with input_variables and template string.
 - Format with specific input.
 - Print the formatted string.

16.2 Flowchart



17 Ollama Equivalent Code

```
1 from langchain_core.prompts import PromptTemplate
2 from langchain_ollama import OllamaLLM
3
4 # Initialize the LLM with Ollama
5 llm = OllamaLLM(model="qwen3:4b")
6
7 prompt = PromptTemplate(
8     input_variables=["input"],
9     template="What is the capital of {input}?",
10 )
11
12 # Create a chain
```

```
13 chain = prompt | llm
14
15 # Invoke the chain
16 result = chain.invoke({"input": "China"})
17 print(result)
```

17.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- prompt=PromptTemplate(...)
- print(prompt.format(input="China"))
```

+++ Added Code

```
+ from langchain_core.prompts import PromptTemplate
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b")
+ chain = prompt | llm
+ result = chain.invoke({"input": "China"})
+ print(result)
```

17.2 Explanation of the Diff

- **Added LLM and chaining:** Introduced OllamaLLM and chain composition with | operator.
- **Changed invocation:** From formatting only to invoking the chain with dict input.
- **Why:** Extends to actual LLM invocation using LangChain's chaining for prompt + model.

18 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 # Initialize the pipeline with Qwen model
4 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
5
6 # Format the prompt
7 prompt = "What is the capital of China?"
8
```

```
9  # Generate text
10 result = pipe(prompt, max_new_tokens=50)
11 print(result[0]['generated_text'])
```

18.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- prompt=PromptTemplate(...)
- print(prompt.format(input="China"))
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt = "What is the capital of China?"
+ result = pipe(prompt, max_new_tokens=50)
+ print(result[0]['generated_text'])
```

18.2 Explanation of the Diff

- **Removed PromptTemplate:** Manual string formatting.
- **Added pipeline generation:** Direct text generation with pipeline.
- **Why:** Hugging Face doesn't use LangChain templates, so manual prompt creation and pipeline invocation.

19 Original Code: code7.py

```
1  from langchain.prompts import PromptTemplate
2
3  from langchain.chat_models import AzureChatOpenAI
4
5  from langchain.chains import LLMChain
6  from dotenv import load_dotenv
7  import os
8  load_dotenv()
9  # Fetch from .env
10 api_key = os.getenv("AZURE_OPENAI_API_KEY")
11 api_base = os.getenv("AZURE_OPENAI_ENDPOINT")
12 deployment_name = os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME")
13 api_version = os.getenv("AZURE_OPENAI_API_VERSION")
14 llm=AzureChatOpenAI(
15     deployment_name=deployment_name,
16     api_version=api_version,
17     api_key=api_key,
18     azure_endpoint=api_base,
19     model_name="gpt-4",
20     temperature=0.9,
21     top_p=0.9,
22     max_tokens=100,
23     # Maximum number of tokens in the response
24 )
25 prompt=PromptTemplate(
26     input_variables=["country"],
27     template="what your opinion on this country {country}?",
28 )
29 chain=LLMChain(llm=llm,prompt=prompt)
30 result=chain.invoke({"country":"India"})
31 print(result['text'])
```

19.1 Explanation of the Code

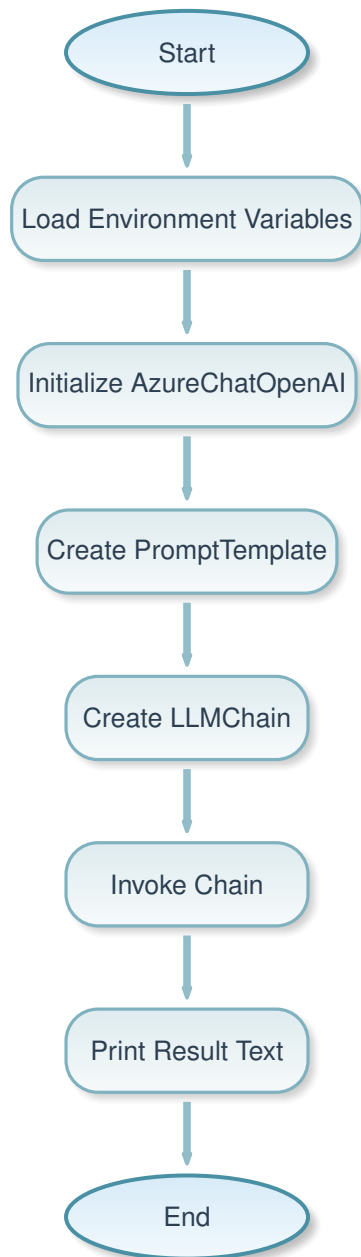
- **What:** This code creates a prompt template, initializes an Azure OpenAI LLM, combines them into an LLMChain, invokes the chain with "India" as input, and prints the generated text.
- **Why:** Demonstrates LangChain's chaining mechanism to combine prompts and LLMs for dynamic query generation. Uses Azure for cloud AI, with parameters like temperature and

top_p for response control.

- **How:**

- Load env vars for Azure credentials.
- Init AzureChatOpenAI with params.
- Create PromptTemplate.
- Create LLMChain with llm and prompt.
- Invoke chain with dict input.
- Print result['text'].

19.2 Flowchart



20 Ollama Equivalent Code

```
1 from langchain_core.prompts import PromptTemplate
2 from langchain_ollama import OllamaLLM
```

```
3
4  # Initialize the LLM with Ollama
5  llm = OllamaLLM(model="qwen3:4b", temperature=0.9)
6
7  prompt = PromptTemplate(
8      input_variables=["country"],
9      template="what your opinion on this country {country}?",
10 )
11
12 # Create a chain
13 chain = prompt | llm
14
15 # Invoke the chain
16 result = chain.invoke({"country": "India"})
17 print(result)
```

20.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- from langchain.chat_models import AzureChatOpenAI
- from langchain.chains import LLMChain
- from dotenv import load_dotenv
- import os
- load_dotenv()
- api_key = os.getenv("AZURE_OPENAI_API_KEY")
- ... (Azure init)
- prompt=PromptTemplate(...)
- chain=LLMChain(llm=llm,prompt=prompt)
- result=chain.invoke({"country":"India"})
- print(result['text'])
```

+++ Added Code

```
+ from langchain_core.prompts import PromptTemplate
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.9)
+ prompt = PromptTemplate(...)
+ chain = prompt | llm
+ result = chain.invoke({"country": "India"})
+ print(result)
```


20.2 Explanation of the Diff

- **Removed Azure setup and old LLMChain:** Eliminated env loading and AzureChatOpenAI.
- **Changed to modern chaining:** Used | operator instead of LLMChain class.
- **Simplified result access:** Direct print of result instead of result['text'].
- **Why:** Updates to newer LangChain syntax and adapts to Ollama local model.

21 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 # Initialize the pipeline with Qwen model
4 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
5
6 # Format the prompt
7 prompt = "what your opinion on this country India?"
8
9 # Generate text
10 result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
11 print(result[0]['generated_text'])
```

21.1 Diff Explanation

— Removed Code

```
- from langchain.prompts import PromptTemplate
- ... (Azure and chain setup)
- result=chain.invoke({"country":"India"})
- print(result['text'])
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt = "what your opinion on this country India?"
+ result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
+ print(result[0]['generated_text'])
```

21.2 Explanation of the Diff

- **Removed LangChain components:** No templates or chains.
- **Manual prompt and pipeline:** Direct string prompt and generation.
- **Why:** Hugging Face focuses on direct model interaction without LangChain abstractions.

22 Original Code: code8.py

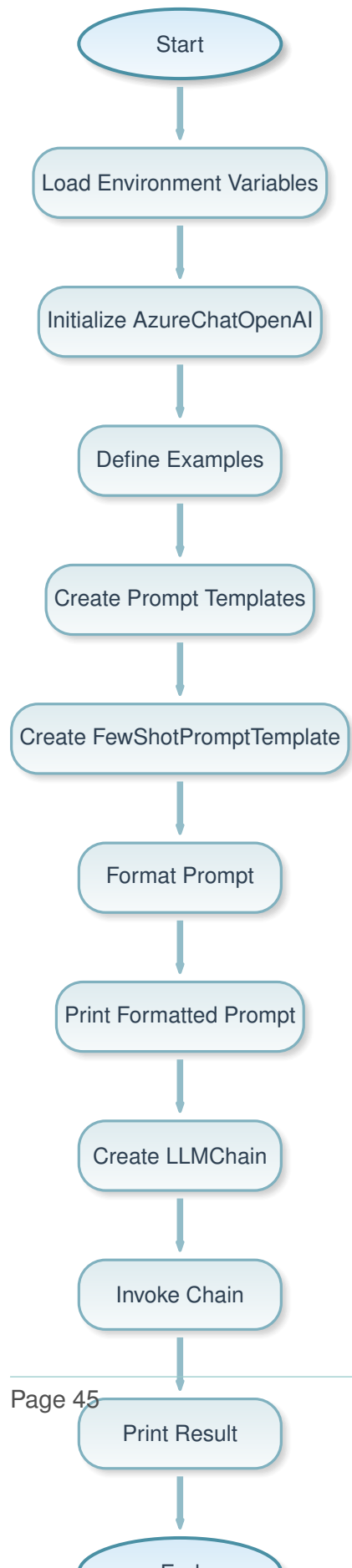
```
1  from langchain.chat_models import AzureChatOpenAI
2  from langchain.prompts import FewShotPromptTemplate, PromptTemplate
3  from dotenv import load_dotenv
4  from langchain.chains import LLMChain
5  import os
6  load_dotenv()
7
8  api_key = os.getenv("AZURE_OPENAI_API_KEY")
9  api_base = os.getenv("AZURE_OPENAI_ENDPOINT")
10 deployment_name = os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME")
11 api_version = os.getenv("AZURE_OPENAI_API_VERSION")
12
13 llm=AzureChatOpenAI(
14     deployment_name=deployment_name,
15     api_version=api_version,
16     api_key=api_key,
17     azure_endpoint=api_base,
18     model_name="gpt-4",
19     temperature=0.9,
20     top_p=0.9,
21     max_tokens=100,
22     # Maximum number of tokens in the response
23 )
24 examples = [
25     {"question": "What is the capital of France?", "answer": "The capital of
26         ↪ France is Paris."},
27     {"question": "What is the capital of Germany?", "answer": "The capital of
28         ↪ Germany is Berlin."},
29     {"question": "What is the capital of Italy?", "answer": "The capital of
30         ↪ Italy is Rome."},
31 ]
32
33 examples_prompt = PromptTemplate(
34     input_variables=["question", "answer"],
35     template="Question: {question}\nAnswer: {answer}",
36 )
37
38 few_shot_prompt = FewShotPromptTemplate(
39     examples=examples,
40     examples_prompt=examples_prompt,
41     input_variables=["question"],
```

```
40     suffix="Answer:",
41     prefix="You are a helpful assistant. Answer the following question.",
42 )
43
44 formatted_prompt = few_shot_prompt.format(question="What is the capital of
    ↪ Spain?")
45 print(formatted_prompt)
46
47 llmchain=LLMChain(
48     llm=llm,
49     prompt=few_shot_prompt,
50 )
51 result=llmchain.invoke({"question":"What is the capital of Spain?"})
52 print(formatted_prompt)
53 print(result['text'])
```

22.1 Explanation of the Code

- **What:** This code sets up a few-shot prompt template with example Q&A pairs, formats it for a new question, prints the formatted prompt, then uses an LLMChain with Azure OpenAI to generate and print the response.
- **Why:** Demonstrates few-shot learning by providing examples in the prompt to guide the model's response. Useful for tasks requiring specific formats or reasoning patterns. Uses Azure for scalable AI.
- **How:**
 - Load env vars.
 - Init AzureChatOpenAI.
 - Define examples list.
 - Create examples_prompt template.
 - Create FewShotPromptTemplate with examples, prefix, suffix.
 - Format the prompt with new question and print.
 - Create LLMChain and invoke with question.
 - Print formatted prompt again and result['text'].

22.2 Flowchart



23 Ollama Equivalent Code

```
1  from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
2  from langchain_ollama import OllamaLLM
3
4  # Initialize the LLM with Ollama
5  llm = OllamaLLM(model="qwen3:4b", temperature=0.9)
6
7  examples = [
8      {"question": "What is the capital of France?", "answer": "The capital
9      ↪ of France is Paris."},
10     {"question": "What is the capital of Germany?", "answer": "The capital
11     ↪ of Germany is Berlin."},
12     {"question": "What is the capital of Italy?", "answer": "The capital
13     ↪ of Italy is Rome."},
14 ]
15
16 examples_prompt = PromptTemplate(
17     input_variables=["question", "answer"],
18     template="Question: {question}\nAnswer: {answer}",
19 )
20
21 few_shot_prompt = FewShotPromptTemplate(
22     examples=examples,
23     example_prompt=examples_prompt,
24     input_variables=["question"],
25     suffix="Answer:",
26     prefix="You are a helpful assistant. Answer the following question.",
27 )
28
29 # Create a chain
30 chain = few_shot_prompt | llm
31
32 # Invoke the chain
33 result = chain.invoke({"question": "What is the capital of Spain?"})
34 print(result)
```

23.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- from langchain.prompts import FewShotPromptTemplate, PromptTemplate
- from dotenv import load_dotenv
- from langchain.chains import LLMChain
- import os
- load_dotenv()
- api_key = ... (Azure setup)
- llm=AzureChatOpenAI(...)
- formatted_prompt = few_shot_prompt.format(question="What is the capital of Spain?")
- print(formatted_prompt)
- llmchain=LLMChain(llm=llm, prompt=few_shot_prompt)
- result=llmchain.invoke({"question":"What is the capital of Spain?"})
- print(formatted_prompt)
- print(result['text'])
```

+++ Added Code

```
+ from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
+ from langchain_ollama import OllamaLLM
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.9)
+ chain = few_shot_prompt | llm
+ result = chain.invoke({"question": "What is the capital of Spain?"})
+ print(result)
```

23.2 Explanation of the Diff

- **Removed Azure and old imports:** Eliminated cloud setup.
- **Updated to modern LangChain:** Used langchain_core and | chaining.
- **Simplified output:** Removed duplicate prints and direct result printing.
- **Why:** Adapts few-shot prompting to Ollama with updated syntax.

24 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 # Initialize the pipeline with Qwen model
```

```

4 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
5
6 # Build the few-shot prompt manually
7 examples = [
8     {"question": "What is the capital of France?", "answer": "The capital
    ↳ of France is Paris."},
9     {"question": "What is the capital of Germany?", "answer": "The capital
    ↳ of Germany is Berlin."},
10    {"question": "What is the capital of Italy?", "answer": "The capital
    ↳ of Italy is Rome."},
11 ]
12
13 prompt = "You are a helpful assistant. Answer the following question.\n"
14 for ex in examples:
15     prompt += f"Question: {ex['question']}\nAnswer: {ex['answer']}\n"
16 prompt += "Question: What is the capital of Spain?\nAnswer:"
17
18 # Generate text
19 result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)
20 print(result[0]['generated_text'])

```

24.1 Diff Explanation

— Removed Code

```

- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain setup)
- formatted_prompt = ...
- print(formatted_prompt)
- llmchain=...
- result=...
- print(...)

```

+++ Added Code

```

+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt = "You are a helpful assistant. Answer the following question.\n"
+ for ex in examples:
+     prompt += f"Question: {ex['question']}\nAnswer: {ex['answer']}\n"
+
+ prompt += "Question: What is the capital of Spain?\nAnswer:"
+ result = pipe(prompt, max_new_tokens=100, temperature=0.9, top_p=0.9)

```



```
+ print(result[0]['generated_text'])
```

24.2 Explanation of the Diff

- **Removed LangChain abstractions:** No templates or chains.
- **Manual prompt construction:** Built the few-shot prompt as a string.
- **Direct pipeline use:** Generated text directly.
- **Why:** Hugging Face requires manual prompt engineering for few-shot learning.

25 Original Code: code9.py

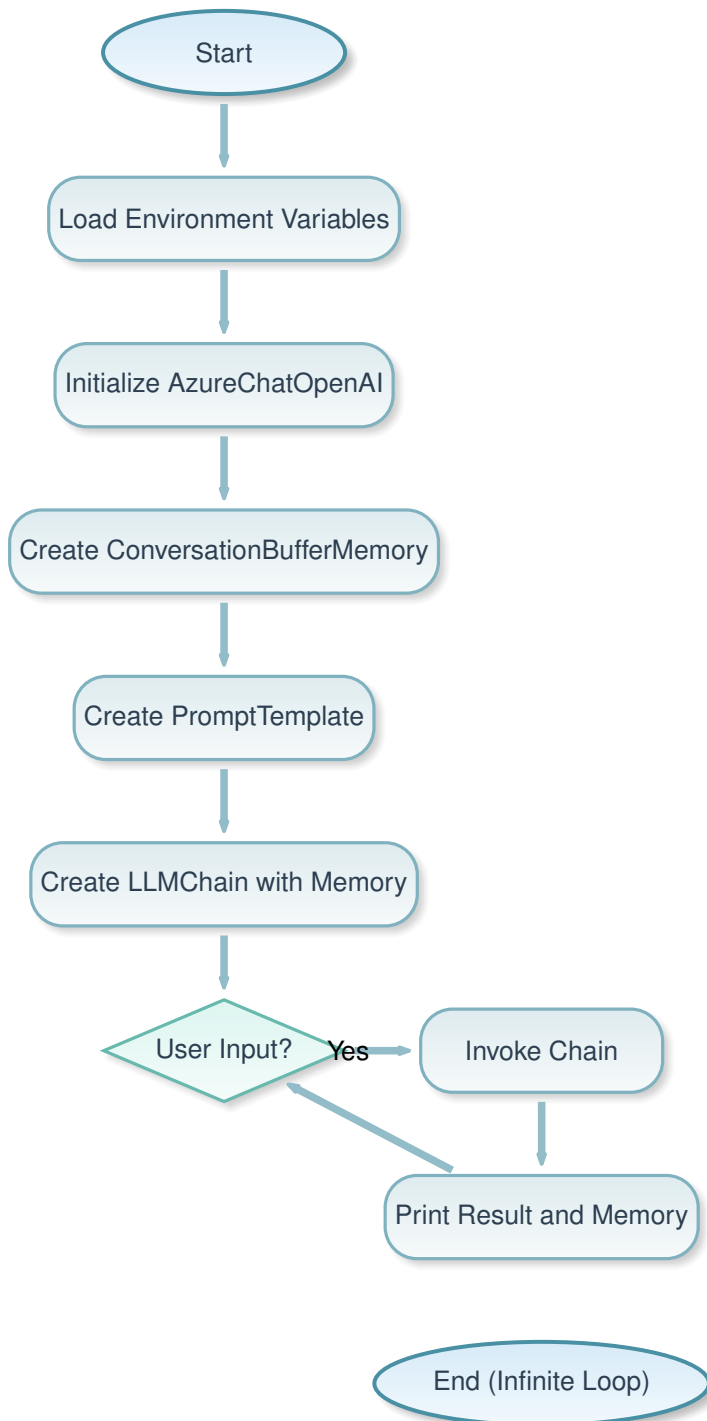
```
1  from langchain.chat_models import AzureChatOpenAI
2  import os
3  from dotenv import load_dotenv
4  load_dotenv()
5  from langchain.chains import LLMChain
6  from langchain.prompts import PromptTemplate
7  from langchain.memory import ConversationBufferMemory
8
9  llm_object = AzureChatOpenAI(
10     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
11     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
12     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
13     deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
14     model_name="gpt-4o",
15     temperature=0.7,
16 )
17
18 memory = ConversationBufferMemory(
19     memory_key="chat_history",
20     return_messages=True,
21 )
22
23 prompt = PromptTemplate(
24     input_variables=["chat_history", "user_input"],
25     template="""
26
27     summary in 50 words technical
28
29     Conversation history:
30     {chat_history}
31     User: {user_input}
32     AI:
33     """
34 )
35
36 chain = LLMChain(
37     llm=llm_object,
38     prompt=prompt,
39     memory=memory
40 )
41
42 while True:
```

```
43     inp = input("Enter the query: ")
44     result = chain.invoke({"user_input": inp})
45     print(result['text'])
46     print(memory.buffer)
```

25.1 Explanation of the Code

- **What:** This code sets up a conversational AI with memory using Azure OpenAI, LangChain's ConversationBufferMemory, and a prompt that includes chat history. It runs in a loop taking user input, generating responses with context, and printing the response and memory buffer.
- **Why:** Demonstrates persistent conversation memory in AI chatbots, allowing the model to recall previous interactions. Useful for coherent multi-turn dialogues. Uses Azure for reliable cloud AI.
- **How:**
 - Load env vars.
 - Init AzureChatOpenAI.
 - Create ConversationBufferMemory.
 - Define prompt with chat_history and user_input.
 - Create LLMChain with llm, prompt, memory.
 - Loop: Get input, invoke chain (memory auto-updates), print result['text'] and memory.buffer.

25.2 Flowchart



26 Ollama Equivalent Code

```
1  from langchain_ollama import OllamaLLM
2  from langchain_core.prompts import PromptTemplate
3  from langchain_classic.memory import ConversationBufferMemory
4
5  llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
6
7  memory = ConversationBufferMemory(
8      memory_key="chat_history",
9      return_messages=True,
10 )
11
12 prompt = PromptTemplate(
13     input_variables=["chat_history", "user_input"],
14     template="""
15
16     summary in 50 words technical
17
18     Conversation history:
19     {chat_history}
20     User: {user_input}
21     AI:
22     """
23 )
24
25 # Using pipe operator instead of deprecated LLMChain
26 chain = prompt | llm
27
28 while True:
29     inp = input("Enter the query: ")
30     result = chain.invoke({"user_input": inp, "chat_history": memory.
31 ↪ buffer})
32     print(result)
33     memory.save_context({"input": inp}, {"output": result})
```

26.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationBufferMemory
- llm_object = AzureChatOpenAI(...)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationBufferMemory
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ chain = prompt | llm
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

26.2 Explanation of the Diff

- **Removed Azure setup:** No env vars or cloud API.
- **Updated imports:** Used langchain_core and langchain_classic for memory.
- **Changed chaining:** | operator instead of LLMChain.
- **Manual memory update:** Since | chain doesn't auto-update memory, save_context is called manually.
- **Why:** Adapts conversational memory to Ollama with modern LangChain, requiring explicit memory management.

27 Hugging Face Equivalent Code

```

1  from transformers import pipeline
2
3  pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5  chat_history = []
6
7  while True:
8      inp = input("Enter the query: ")
9      history_str = "\n".join(chat_history)
10     prompt = f"""
11
12     "summary in 50 words technical
13
14     Conversation history:
15     {history_str}
16     User: {inp}
17     AI:
18     """
19     result = pipe(prompt, max_new_tokens=100, temperature=0.7,
20     ↪ num_return_sequences=1)
21     response = result[0]['generated_text'].split("AI:")[1].strip() if "AI"
22     ↪ ":" in result[0]['generated_text'] else result[0]['generated_text']
23     print(response)
24     chat_history.append(f"User: {inp}")
25     chat_history.append(f"AI: {response}")
26     print(chat_history)

```

27.1 Diff Explanation

— Removed Code

```

- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain components)
- memory = ConversationBufferMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)

```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []
+ history_str = "\n".join(chat_history)
+ prompt = f"..." (manual prompt building)
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI:")[1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}")
+ chat_history.append(f"AI: {response}")
+ print(chat_history)
```

27.2 Explanation of the Diff

- **Removed LangChain memory:** No built-in memory management.
- **Manual history tracking:** Used list to store and build prompt.
- **Prompt engineering:** Constructed prompt string manually with history.
- **Response parsing:** Extracted response from generated text.
- **Why:** Hugging Face requires custom implementation for conversational memory, as it lacks LangChain's abstractions.

28 Original Code: code9A.py

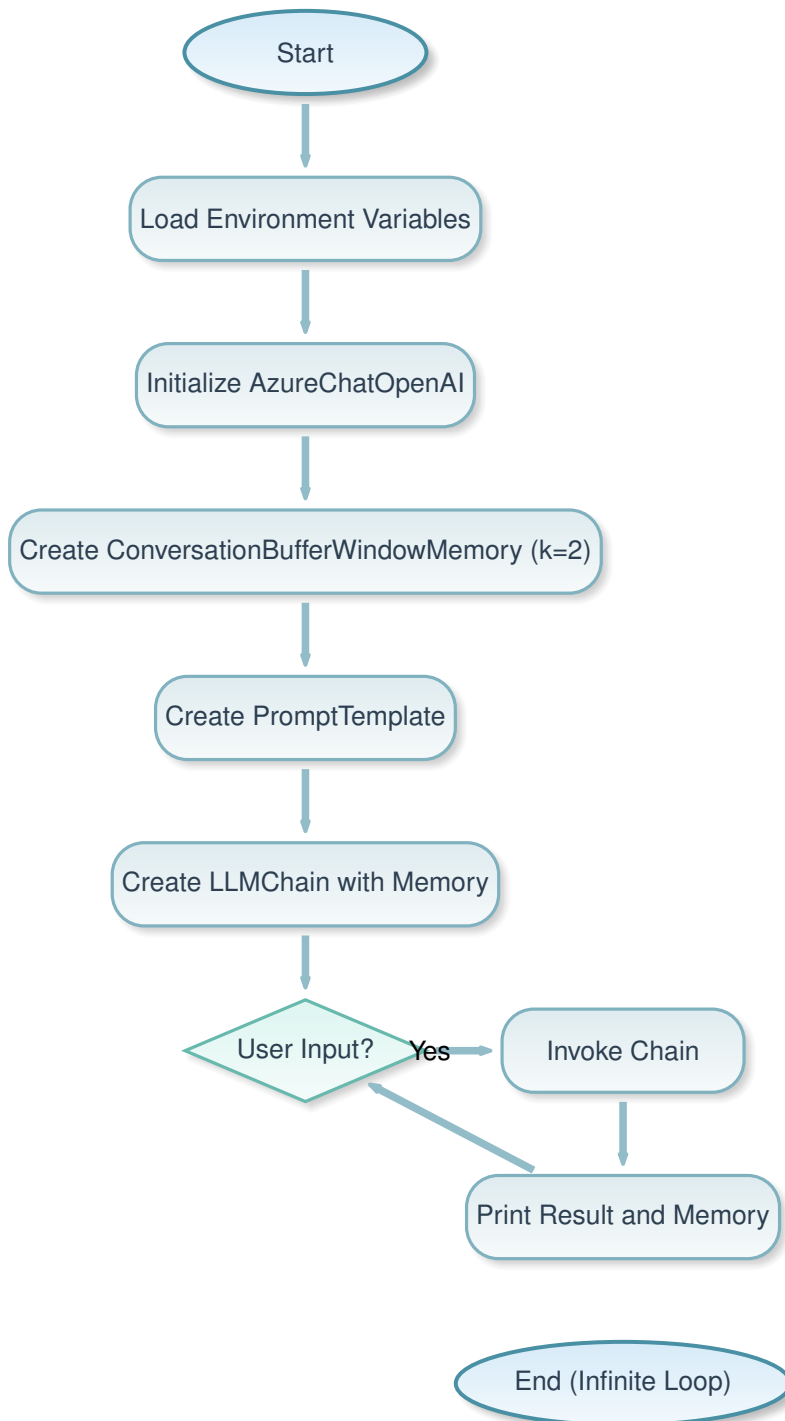
```
1  from langchain.chat_models import AzureChatOpenAI
2  import os
3  from dotenv import load_dotenv
4  load_dotenv()
5  from langchain.chains import LLMChain
6  from langchain.prompts import PromptTemplate
7  from langchain.memory import ConversationBufferWindowMemory
8
9  llm_object = AzureChatOpenAI(
10     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
11     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
12     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
13     deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
14     model_name="gpt-4o",
15     temperature=0.7,
16 )
17
18 memory = ConversationBufferWindowMemory(
19     memory_key="chat_history",
20     return_messages=True,
21     k=2
22 )
23
24 prompt = PromptTemplate(
25     input_variables=["chat_history", "user_input"],
26     template="""
27
28     summary in 50 words technical
29
30     Conversation history:
31     {chat_history}
32     User: {user_input}
33     AI:
34     """
35 )
36
37 chain = LLMChain(
38     llm=llm_object,
39     prompt=prompt,
40     memory=memory
41 )
42
```

```
43 while True:
44     inp = input("Enter the query: ")
45     result = chain.invoke({"user_input": inp})
46     print(result['text'])
47     print(memory.buffer)
```

28.1 Explanation of the Code

- **What:** This code implements a conversational AI with windowed memory (keeping only the last 2 interactions) using Azure OpenAI and LangChain's ConversationBufferWindowMemory. It loops for user input, generates context-aware responses, and prints the result and memory buffer.
- **Why:** Demonstrates memory management to prevent context from growing indefinitely, focusing on recent history for efficiency. Useful for long conversations where old context becomes less relevant. Uses Azure for consistent AI performance.
- **How:**
 - Load env vars.
 - Init AzureChatOpenAI.
 - Create ConversationBufferWindowMemory with k=2 (last 2 turns).
 - Define prompt with history and input.
 - Create LLMChain with memory.
 - Loop: Input, invoke, print result['text'] and memory.buffer.

28.2 Flowchart



29 Ollama Equivalent Code

```
1  from langchain_ollama import OllamaLLM
2  from langchain_core.prompts import PromptTemplate
3  from langchain_classic.memory import ConversationBufferWindowMemory
4
5  llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
6
7  memory = ConversationBufferWindowMemory(
8      memory_key="chat_history",
9      return_messages=True,
10     k=2
11 )
12
13 prompt = PromptTemplate(
14     input_variables=["chat_history", "user_input"],
15     template="""
16
17     summary in 50 words technical
18
19     Conversation history:
20     {chat_history}
21     User: {user_input}
22     AI:
23     """
24 )
25
26 # Using pipe operator instead of deprecated LLMChain
27 chain = prompt | llm_object
28
29 while True:
30     inp = input("Enter the query: ")
31     result = chain.invoke({"user_input": inp, "chat_history": memory.
32 ↪ buffer})
33     print(result)
34     memory.save_context({"input": inp}, {"output": result})
```

29.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationBufferWindowMemory
- llm_object = AzureChatOpenAI(...)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationBufferWindowMemory
+ llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ chain = prompt | llm_object
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

29.2 Explanation of the Diff

- **Removed Azure setup:** Local Ollama instead.
- **Updated imports:** langchain_core and langchain_classic.
- **Modern chaining:** | operator.
- **Manual memory save:** Required for | chains.
- **Why:** Adapts windowed memory to Ollama with updated LangChain patterns.

30 Hugging Face Equivalent Code

```
1 from transformers import pipeline
```

```

2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 chat_history = []
6
7 while True:
8     inp = input("Enter the query: ")
9     # Keep only last 2 exchanges (4 messages: 2 user + 2 AI)
10    if len(chat_history) > 4:
11        chat_history = chat_history[-4:]
12    history_str = "\n".join(chat_history)
13    prompt = f"""
14
15    "summary in 50 words technical
16
17    Conversation history:
18    {history_str}
19    User: {inp}
20    AI:
21    """
22    result = pipe(prompt, max_new_tokens=100, temperature=0.7,
23    ↪ num_return_sequences=1)
24    response = result[0]['generated_text'].split("AI:")[-1].strip() if "AI
25    ↪ :" in result[0]['generated_text'] else result[0]['generated_text']
26    print(response)
27    chat_history.append(f"User: {inp}")
28    chat_history.append(f"AI: {response}")
29    print(chat_history)

```

30.1 Diff Explanation

— Removed Code

```

- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- memory = ConversationBufferWindowMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)

```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []
+ if len(chat_history) > 4: chat_history = chat_history[-4:]
+ history_str = "\n".join(chat_history)
+ prompt = f"... (manual)"
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI: ")[-1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}")
+ chat_history.append(f"AI: {response}")
+ print(chat_history)
```

30.2 Explanation of the Diff

- **Removed LangChain memory:** Custom window implementation.
- **Manual windowing:** Truncate history to last 4 messages.
- **Prompt building:** String concatenation for history.
- **Why:** Hugging Face requires manual memory and prompt management for windowed conversations.

31 Original Code: code9B.py

```
1  from langchain.chat_models import AzureChatOpenAI
2  import os
3  from dotenv import load_dotenv
4  load_dotenv()
5  from langchain.chains import LLMChain
6  from langchain.prompts import PromptTemplate
7  from langchain.memory import ConversationSummaryMemory
8
9  llm_object = AzureChatOpenAI(
10     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
11     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
12     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
13     deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
14     model_name="gpt-4o",
15     temperature=0.7,
16 )
17
18 memory = ConversationSummaryMemory(
19     memory_key="chat_history",
20     return_messages=True,
21     llm=llm
22 )
23
24 prompt = PromptTemplate(
25     input_variables=["chat_history", "user_input"],
26     template="""
27
28     summary in 50 words technical
29
30     Conversation history:
31     {chat_history}
32     User: {user_input}
33     AI:
34     """
35 )
36
37 chain = LLMChain(
38     llm=llm_object,
39     prompt=prompt,
40     memory=memory
41 )
42
```

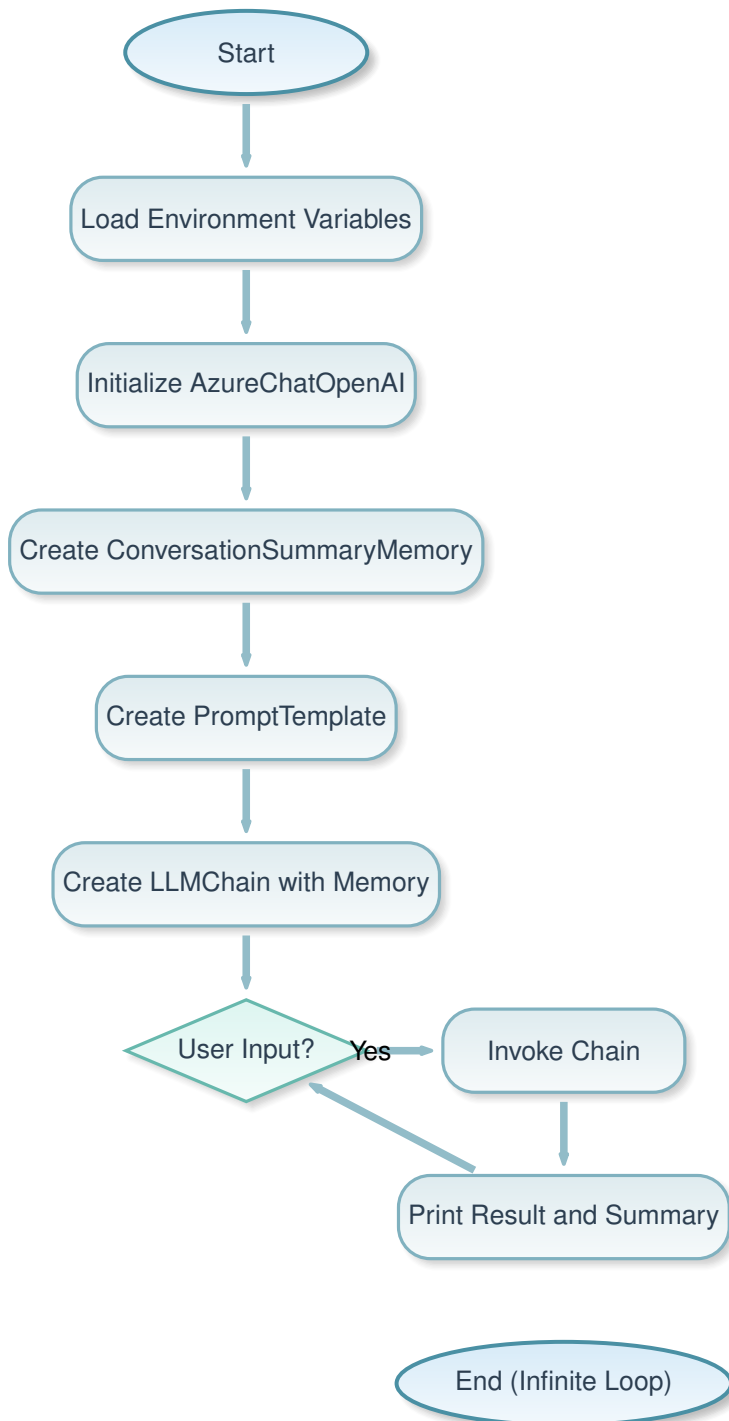


```
43 while True:
44     inp = input("Enter the query: ")
45     result = chain.invoke({"user_input": inp})
46     print(result['text'])
47     print(memory.buffer)
```

31.1 Explanation of the Code

- **What:** This code uses ConversationSummaryMemory to maintain a summarized version of the conversation history, condensing past interactions into summaries. It employs Azure OpenAI in a loop for user queries, printing responses and the current memory summary.
- **Why:** Summary memory prevents context from growing too large by summarizing old parts, maintaining efficiency in long conversations. Useful when full history isn't needed but key points are. Leverages Azure for summarization and response generation.
- **How:**
 - Load env vars.
 - Init AzureChatOpenAI (note: llm in memory should be llm_object).
 - Create ConversationSummaryMemory with llm for summarization.
 - Define prompt with history and input.
 - Create LLMChain with memory.
 - Loop: Input, invoke, print result['text'] and memory.buffer (summary).

31.2 Flowchart



32 Ollama Equivalent Code

```
1  from langchain_ollama import OllamaLLM
2  from langchain_core.prompts import PromptTemplate
3  from langchain_classic.memory import ConversationSummaryMemory
4
5  llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
6
7  memory = ConversationSummaryMemory(
8      memory_key="chat_history",
9      return_messages=True,
10     llm=llm_object
11 )
12
13 prompt = PromptTemplate(
14     input_variables=["chat_history", "user_input"],
15     template="""
16
17     summary in 50 words technical
18
19     Conversation history:
20     {chat_history}
21     User: {user_input}
22     AI:
23     """
24 )
25
26 # Using pipe operator instead of deprecated LLMChain
27 chain = prompt | llm_object
28
29 while True:
30     inp = input("Enter the query: ")
31     result = chain.invoke({"user_input": inp, "chat_history": memory.
32 ↪ buffer})
33     print(result)
34     memory.save_context({"input": inp}, {"output": result})
```

32.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationSummaryMemory
- llm_object = AzureChatOpenAI(...)
- memory = ConversationSummaryMemory(..., llm=llm)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationSummaryMemory
+ llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ memory = ConversationSummaryMemory(..., llm=llm_object)
+ chain = prompt | llm_object
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

32.2 Explanation of the Diff

- **Removed Azure:** Local Ollama.
- **Updated imports:** langchain_core and langchain_classic.
- **Modern chaining:** | operator.
- **Manual save:** For memory update.
- **Why:** Adapts summary memory to Ollama, using the same LLM for summarization.

33 Hugging Face Equivalent Code

```

1  from transformers import pipeline
2
3  pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5  chat_history = []
6
7  while True:
8      inp = input("Enter the query: ")
9      history_str = "\n".join(chat_history)
10     prompt = f"""
11
12     summary in 50 words technical
13
14     Conversation history:
15     {history_str}
16     User: {inp}
17     AI:
18     """
19     result = pipe(prompt, max_new_tokens=100, temperature=0.7,
20     ↪ num_return_sequences=1)
21     response = result[0]['generated_text'].split("AI:")[1].strip() if "AI"
22     ↪ ":" in result[0]['generated_text'] else result[0]['generated_text']
23     print(response)
24     chat_history.append(f"User: {inp}")
25     chat_history.append(f"AI: {response}")
26     print(chat_history)

```

33.1 Diff Explanation

— Removed Code

```

- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- memory = ConversationSummaryMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)

```

+++ Added Code

```

+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ chat_history = []

```

```
+ history_str = "\n".join(chat_history)
+ prompt = f"..." (manual)
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)
+ response = result[0]['generated_text'].split("AI:")[1].strip() ...
+ print(response)
+ chat_history.append(f"User: {inp}")
+ chat_history.append(f"AI: {response}")
+ print(chat_history)
```

33.2 Explanation of the Diff

- **Removed summary memory:** No summarization, full history.
- **Manual history:** List-based tracking.
- **Why:** Hugging Face lacks built-in summary memory, so simplified to buffer-like behavior.

34 Original Code: code9C.py

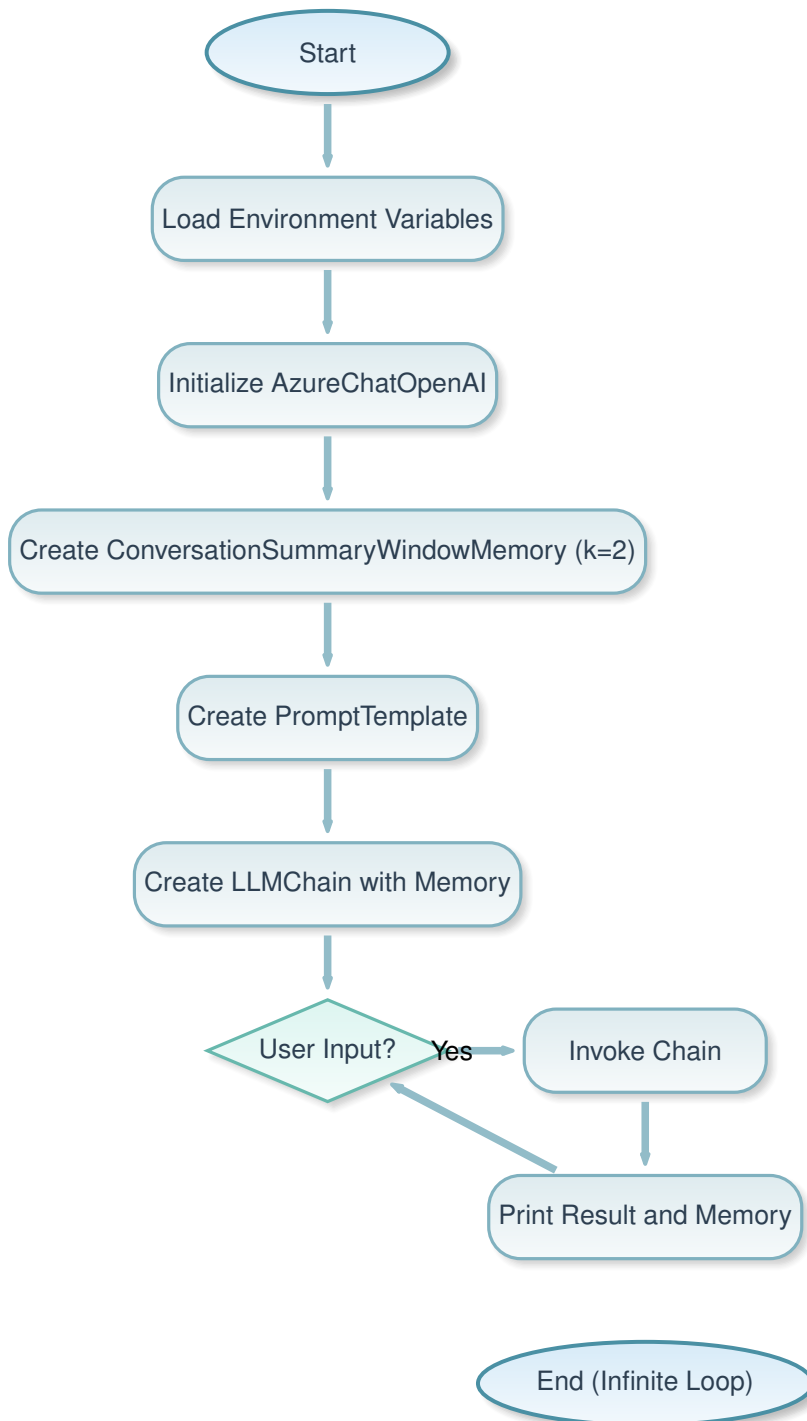
```
1  from langchain.chat_models import AzureChatOpenAI
2  import os
3  from dotenv import load_dotenv
4  load_dotenv()
5  from langchain.chains import LLMChain
6  from langchain.prompts import PromptTemplate
7  from langchain.memory import ConversationSummaryWindowMemory
8
9  llm_object = AzureChatOpenAI(
10     openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
11     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
12     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
13     deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
14     model_name="gpt-4o",
15     temperature=0.7,
16 )
17
18 memory = ConversationSummaryWindowMemory(
19     memory_key="chat_history",
20     return_messages=True,
21     llm=llm,
22     k=2
23 )
24
25 prompt = PromptTemplate(
26     input_variables=["chat_history", "user_input"],
27     template="""
28
29     summary in 50 words technical
30
31     Conversation history:
32     {chat_history}
33     User: {user_input}
34     AI:
35     """
36 )
37
38 chain = LLMChain(
39     llm=llm_object,
40     prompt=prompt,
41     memory=memory
42 )
```

```
43
44 while True:
45     inp = input("Enter the query: ")
46     result = chain.invoke({"user_input": inp})
47     print(result['text'])
48     print(memory.buffer)
```

34.1 Explanation of the Code

- **What:** This code employs ConversationSummaryWindowMemory, which summarizes the conversation but keeps the last k interactions in full. It uses Azure OpenAI in a loop, printing responses and the summarized/windowed memory.
- **Why:** Combines summarization for old parts with full retention of recent interactions, balancing efficiency and recency. Ideal for conversations needing recent context but compressed history. Uses Azure for both generation and summarization.
- **How:**
 - Load env vars.
 - Init AzureChatOpenAI.
 - Create ConversationSummaryWindowMemory with llm (should be llm_object) and k=2.
 - Define prompt.
 - Create LLMChain.
 - Loop: Input, invoke, print result['text'] and memory.buffer.

34.2 Flowchart



35 Ollama Equivalent Code

```
1  from langchain_ollama import OllamaLLM
2  from langchain_core.prompts import PromptTemplate
3  from langchain_classic.memory import ConversationSummaryBufferMemory
4
5  llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
6
7  memory = ConversationSummaryBufferMemory(
8      memory_key="chat_history",
9      return_messages=True,
10     llm=llm_object,
11 )
12
13 prompt = PromptTemplate(
14     input_variables=["chat_history", "user_input"],
15     template="""
16
17     summary in 50 words technical
18
19     Conversation history:
20     {chat_history}
21     User: {user_input}
22     AI:
23     """
24 )
25
26 # Using pipe operator instead of deprecated LLMChain
27 chain = prompt | llm_object
28
29 while True:
30     inp = input("Enter the query: ")
31     result = chain.invoke({"user_input": inp, "chat_history": memory.
32 ↪ buffer})
33     print(result)
34     memory.save_context({"input": inp}, {"output": result})
```

35.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- from langchain.memory import ConversationSummaryWindowMemory
- llm_object = AzureChatOpenAI(...)
- memory = ConversationSummaryWindowMemory(..., llm=llm, k=2)
- chain = LLMChain(llm=llm_object, prompt=prompt, memory=memory)
- result = chain.invoke({"user_input": inp})
- print(result['text'])
- print(memory.buffer)
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ from langchain_classic.memory import ConversationSummaryBufferMemory
+ llm_object = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ memory = ConversationSummaryBufferMemory(..., llm=llm_object)
+ chain = prompt | llm_object
+ result = chain.invoke({"user_input": inp, "chat_history": memory.buffer})
+ print(result)
+ memory.save_context({"input": inp})
```

35.2 Explanation of the Diff

- **Removed Azure:** Local Ollama.
- **Changed memory type:** ConversationSummaryBufferMemory instead of Window, as Window may not be available.
- **Modern chaining:** | operator.
- **Why:** Adapts to available memory types in langchain_classic, using buffer with summarization.

36 Hugging Face Equivalent Code

```

1  from transformers import pipeline
2
3  pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5  chat_history = []
6
7  while True:
8      inp = input("Enter the query: ")
9      # Approximate: keep last 4 messages
10     if len(chat_history) > 4:
11         chat_history = chat_history[-4:]
12     history_str = "\n".join(chat_history)
13     prompt = f"""
14
15     "summary in 50 words technical
16
17     Conversation history:
18     {history_str}
19     User: {inp}
20     AI:
21     """
22     result = pipe(prompt, max_new_tokens=100, temperature=0.7,
23     ↪ num_return_sequences=1)
24     response = result[0]['generated_text'].split("AI:")[-1].strip() if "AI
25     ↪ : " in result[0]['generated_text'] else result[0]['generated_text']
26     print(response)
27     chat_history.append(f"User: {inp}")
28     chat_history.append(f"AI: {response}")
29     print(chat_history)

```

36.1 Diff Explanation

— Removed Code

```

- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- memory = ConversationSummaryWindowMemory(...)
- chain = LLMChain(...)
- result = chain.invoke({"user_input": inp})

```

```
- print(result['text'])  
- print(memory.buffer)
```

+++ Added Code

```
+ from transformers import pipeline  
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")  
+ chat_history = []  
+ if len(chat_history) > 4: chat_history = chat_history[-4:]  
+ history_str = "\n".join(chat_history)  
+ prompt = f"... (manual)"  
+ result = pipe(prompt, max_new_tokens=100, temperature=0.7, num_return_sequences=1)  
+ response = result[0]['generated_text'].split("AI: ")[-1].strip() ...  
+ print(response)  
+ chat_history.append(f"User: {inp}")  
+ chat_history.append(f"AI: {response}")  
+ print(chat_history)
```

36.2 Explanation of the Diff

- **Removed summary window memory:** Manual windowing without summarization.
- **Why:** Hugging Face doesn't have built-in summary memory, so approximated with buffer window.

37 Original Code: code10.py

```
1  #component 1 LLM
2  from langchain.chat_models import AzureChatOpenAI
3  import os
4  from dotenv import load_dotenv
5  load_dotenv()
6  llm = AzureChatOpenAI(
7      openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
8      openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
9      openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
10     deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
11     model_name="gpt-4o",
12     temperature=0.7,
13 )
14
15
16 from langchain.chains import LLMChain
17
18 #component 2 PROMPT
19 from langchain.prompts import PromptTemplate
20 prompt=PromptTemplate(
21     template="what is the capital of {input}"
22 )
23
24 pipeline=LLMChain(
25     llm=llm,
26     prompt=prompt
27 )
28
29
30 result=pipeline.invoke(input="china")
31
32 print(result['text'])
```

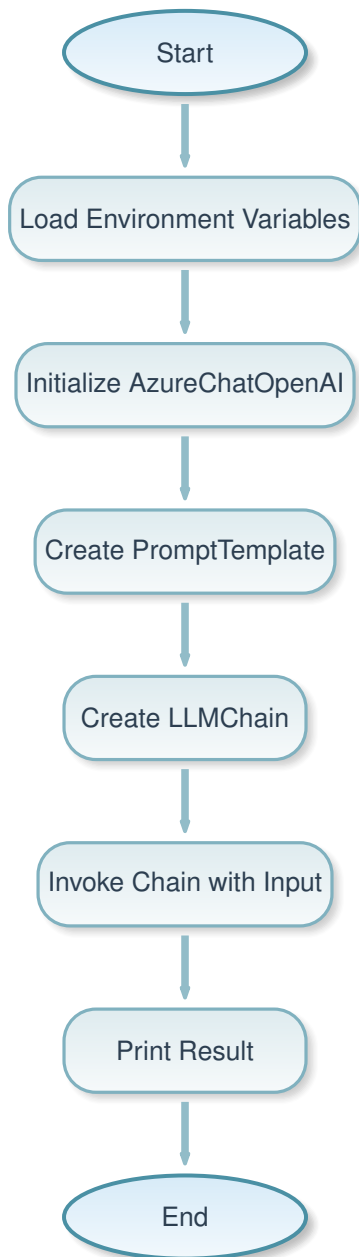
37.1 Explanation of the Code

- **What:** This code creates a simple pipeline using LLMChain that combines a prompt template with Azure OpenAI LLM, invokes it with "china" as input, and prints the response.
- **Why:** Demonstrates basic chaining of prompt and LLM in LangChain for templated queries. Uses Azure for cloud-based AI, with env vars for security.

- **How:**

- Load env vars.
- Init AzureChatOpenAI.
- Create PromptTemplate with placeholder.
- Create LLMChain with llm and prompt.
- Invoke with dict input.
- Print result['text'].

37.2 Flowchart



38 Ollama Equivalent Code

```
1 from langchain_ollama import OllamaLLM
2 from langchain_core.prompts import PromptTemplate
```



```
3
4 llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
5
6 prompt = PromptTemplate(
7     template="what is the capital of {input}"
8 )
9
10 # Using pipe operator instead of deprecated LLMChain
11 pipeline = prompt | llm
12
13 result = pipeline.invoke({"input": "china"})
14
15 print(result)
```

38.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- llm = AzureChatOpenAI(...)
- from langchain.chains import LLMChain
- from langchain.prompts import PromptTemplate
- prompt=PromptTemplate(...)
- pipeline=LLMChain(llm=llm, prompt=prompt)
- result=pipeline.invoke(input="china")
- print(result['text'])
```

+++ Added Code

```
+ from langchain_ollama import OllamaLLM
+ from langchain_core.prompts import PromptTemplate
+ llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ prompt = PromptTemplate(...)
+ pipeline = prompt | llm
+ result = pipeline.invoke({"input": "china"})
+ print(result)
```

38.2 Explanation of the Diff

- **Removed Azure setup:** Local Ollama.
- **Updated imports:** langchain_core.

- **Modern chaining:** | instead of LLMChain.
- **Why:** Simplifies to local execution with updated LangChain syntax.

39 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2
3 pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
4
5 prompt_template = "what is the capital of {input}"
6
7 input_value = "china"
8 prompt = prompt_template.format(input=input_value)
9
10 result = pipe(prompt, max_new_tokens=50, temperature=0.7,
11               ↪ num_return_sequences=1)
12 print(result[0]['generated_text'])
```

39.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- pipeline=LLMChain(...)
- result=pipeline.invoke(input="china")
- print(result['text'])
```

+++ Added Code

```
+ from transformers import pipeline
+ pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
+ prompt_template = "what is the capital of {input}"
+ input_value = "china"
+ prompt = prompt_template.format(input=input_value)
+ result = pipe(prompt, max_new_tokens=50, temperature=0.7, num_return_sequences=1)
+ print(result[0]['generated_text'])
```

39.2 Explanation of the Diff

- **Removed LangChain:** Manual template formatting.
- **Direct pipeline:** No chaining abstraction.
- **Why:** Hugging Face requires explicit prompt construction.

40 Original Code: code11.py

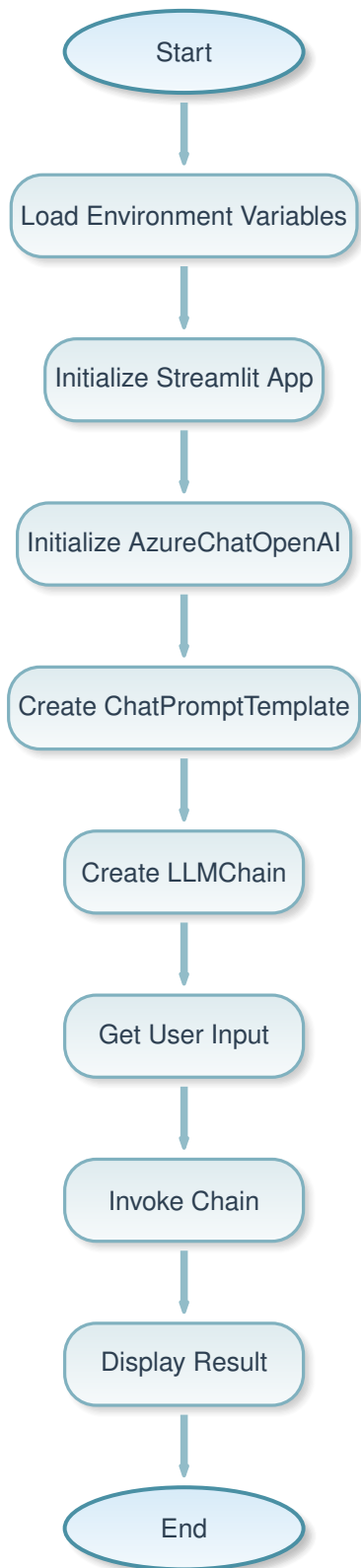
```
1  # Component -1
2  from langchain.chat_models import AzureChatOpenAI
3  import os
4  from dotenv import load_dotenv
5  load_dotenv()
6  import streamlit as st
7  st.title("MY FIRST CHATBOT")
8  my_llm = AzureChatOpenAI(
9      openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
10     openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
11     openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
12     deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
13     model_name="gpt-4o",
14     temperature=0.7,
15 )
16
17 # Component -2 (ChatPromptTemplate Version)
18 from langchain.prompts import ChatPromptTemplate
19
20 my_prompt = ChatPromptTemplate.from_messages([
21     ("system", "You are a helpful assistant."),
22     ("human", "{input}")
23 ])
24
25 from langchain.chains import LLMChain
26
27 my_pipeline = LLMChain(
28     llm=my_llm,
29     prompt=my_prompt
30 )
31 usertext=st.text_input("USER:")
32 result = my_pipeline.invoke({"input": usertext})
33 st.write(result["text"])
```

40.1 Explanation of the Code

- **What:** This code builds a Streamlit web app for a chatbot, using ChatPromptTemplate with system and human messages, combined with Azure OpenAI in an LLMChain, taking user input and displaying the AI response.

- **Why:** Demonstrates creating a simple web-based chatbot with LangChain's chat prompt templates for structured conversations. Uses Streamlit for UI and Azure for AI.
- **How:**
 - Load env vars.
 - Init Streamlit title.
 - Init AzureChatOpenAI.
 - Create ChatPromptTemplate with messages.
 - Create LLMChain.
 - Get user input via `st.text_input`.
 - Invoke chain and display result["text"].

40.2 Flowchart



41 Ollama Equivalent Code

```

1  # Component -1
2  from langchain_ollama import OllamaLLM
3  import streamlit as st
4  st.title("MY FIRST CHATBOT")
5  my_llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
6
7  # Component -2 (ChatPromptTemplate Version)
8  from langchain_core.prompts import ChatPromptTemplate
9
10 my_prompt = ChatPromptTemplate.from_messages([
11     ("system", "You are a helpful assistant."),
12     ("human", "{input}")
13 ])
14
15 # Using pipe operator instead of deprecated LLMChain
16 my_pipeline = my_prompt | my_llm
17 usertext=st.text_input("USER:")
18 result = my_pipeline.invoke({"input": usertext})
19 st.write(result["text"])

```

41.1 Diff Explanation

— Removed Code

```

- from langchain.chat_models import AzureChatOpenAI
- import os
- from dotenv import load_dotenv
- load_dotenv()
- my_llm = AzureChatOpenAI(...)
- from langchain.prompts import ChatPromptTemplate
- from langchain.chains import LLMChain
- my_pipeline = LLMChain(llm=my_llm, prompt=my_prompt)

```

+++ Added Code

```

+ from langchain_ollama import OllamaLLM
+ my_llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
+ from langchain_core.prompts import ChatPromptTemplate
+ my_pipeline = my_prompt | my_llm

```

41.2 Explanation of the Diff

- **Removed Azure:** Local Ollama.
- **Updated imports:** langchain_core.
- **Modern chaining:** | operator.
- **Why:** Adapts to local model with updated syntax.

42 Hugging Face Equivalent Code

```
1 from transformers import pipeline
2 import streamlit as st
3 st.title("MY FIRST CHATBOT")
4
5 generator = pipeline("text-generation", model="gpt2")
6
7 usertext = st.text_input("USER:")
8 if usertext:
9     prompt = f"You are a helpful assistant. {usertext}"
10    result = generator(prompt, max_length=50, num_return_sequences=1)
11    st.write(result[0]['generated_text'])
```

42.1 Diff Explanation

— Removed Code

```
- from langchain.chat_models import AzureChatOpenAI
- ... (all LangChain)
- my_pipeline = LLMChain(...)
- result = my_pipeline.invoke({"input": usertext})
- st.write(result["text"])
```

+++ Added Code

```
+ from transformers import pipeline
+ generator = pipeline("text-generation", model="gpt2")
+ usertext = st.text_input("USER:")
+ if usertext:
+     prompt = f"You are a helpful assistant. {usertext}"
```



```
+     result = generator(prompt, max_length=50, num_return_sequences=1)
+     st.write(result[0]['generated_text'])
```

42.2 Explanation of the Diff

- **Removed LangChain:** Manual prompt construction.
- **Changed model:** Uses gpt2 instead of Qwen.
- **Why:** Hugging Face requires direct pipeline usage and prompt engineering.

43 Original Code: code12.py

```
1  # In a RAG pipeline you must:
2
3  # 1. Collect documents (PDFs, text files, DOCX, website pages, database
       ↪ rows, etc.)
4
5
6  # 2. Load them into your program
7
8
9  # 3. Split them into chunks
10
11
12 # 4. Convert them into embeddings
13
14
15 # 5. Store them in a vector database
16
17
18
19 # Here we focus on Step 2 -- Loading documents.
20
21
22 # ---
23
24 # Most Common Ways to Load Documents for RAG
25
26 # Here are the most frequently used loaders in Python (LangChain):
27
28
29 # ---
30
31 # Load PDFs
32
33 # Using LangChain's PyPDFLoader
34
35 from langchain.document_loaders import PyPDFLoader
36
37 loader = PyPDFLoader("sample.pdf")
38 documents = loader.load()
39
40 print(documents)
41
```

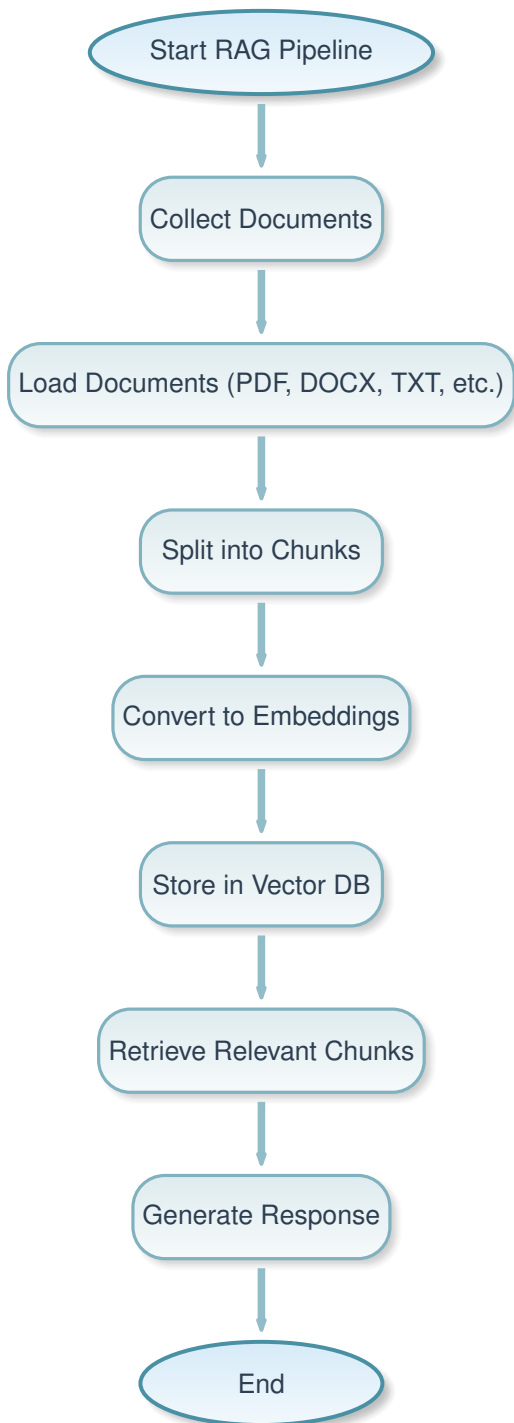
```
42
43
44
45 # Load DOCX / Word files
46
47 from langchain.document_loaders import Docx2txtLoader
48
49 loader = Docx2txtLoader("sample.docx")
50 documents = loader.load()
51
52
53
54 # Load Text Files (.txt)
55
56 from langchain.document_loaders import TextLoader
57
58 loader = TextLoader("notes.txt")
59 documents = loader.load()
60
61
62 # Load Multiple Documents in a Folder
63
64 from langchain.document_loaders import DirectoryLoader
65
66 loader = DirectoryLoader(
67     "data/",
68     glob="*/.pdf",
69     show_progress=True
70 )
71
72 documents = loader.load()
73
74
75 Load Website Pages (HTML)
76
77 from langchain.document_loaders import WebBaseLoader
78
79 loader = WebBaseLoader("http://shooliniuniversity.com/")
80
81 documents = loader.load()
82
83
84 # Load Markdown Files (.md)
85
86 from langchain.document_loaders import UnstructuredMarkdownLoader
87
88 loader = UnstructuredMarkdownLoader("readme.md")
```

```
89 documents = loader.load()
90
91
92 # Load CSV Files
93
94 from langchain.document_loaders.csv_loader import CSVLoader
95
96 loader = CSVLoader("data.csv")
97 documents = loader.load()
```

43.1 Explanation of the Code

- **What:** This code provides examples of various document loaders in LangChain for RAG (Retrieval-Augmented Generation) systems, including loaders for PDFs, DOCX, text files, directories, websites, Markdown, and CSV files.
- **Why:** Document loading is a crucial step in RAG pipelines to ingest diverse data sources into the system for later chunking, embedding, and retrieval. LangChain provides unified interfaces for different file types to simplify data ingestion.
- **How:**
 - Import specific loader classes from `langchain.document_loaders`.
 - Instantiate loader with file path or URL.
 - Call `load()` method to get documents (list of Document objects).
 - Print or process the loaded documents.

43.2 Flowchart



44 Ollama Equivalent Code

```
1  # In a RAG pipeline you must:
2
3  # 1. Collect documents (PDFs, text files, DOCX, website pages, database
       ↪ rows, etc.)
4
5
6  # 2. Load them into your program
7
8
9  # 3. Split them into chunks
10
11
12 # 4. Convert them into embeddings
13
14
15 # 5. Store them in a vector database
16
17
18
19 # Here we focus on Step 2 -- Loading documents.
20
21
22 # ---
23
24 # Most Common Ways to Load Documents for RAG
25
26 # Here are the most frequently used loaders in Python (LangChain):
27
28
29 # ---
30
31 # Load PDFs
32
33 # Using LangChain's PyPDFLoader
34
35 from langchain_community.document_loaders import PyPDFLoader
36
37 loader = PyPDFLoader("sample.pdf")
38 documents = loader.load()
39
40 print(documents)
41
```

```
42
43
44
45 # Load DOCX / Word files
46
47 from langchain_community.document_loaders import Docx2txtLoader
48
49 loader = Docx2txtLoader("sample.docx")
50 documents = loader.load()
51
52
53
54 # Load Text Files (.txt)
55
56 from langchain_community.document_loaders import TextLoader
57
58 loader = TextLoader("notes.txt")
59 documents = loader.load()
60
61
62 # Load Multiple Documents in a Folder
63
64 from langchain_community.document_loaders import DirectoryLoader
65
66 loader = DirectoryLoader(
67     "data/",
68     glob="*/.pdf",
69     show_progress=True
70 )
71
72 documents = loader.load()
73
74
75 # Load Website Pages (HTML)
76
77 from langchain_community.document_loaders import WebBaseLoader
78
79 loader = WebBaseLoader("http://shooliniuniversity.com/")
80 documents = loader.load()
81
82
83 # Load Markdown Files (.md)
84
85 from langchain_community.document_loaders import
86     ↪ UnstructuredMarkdownLoader
87
88 loader = UnstructuredMarkdownLoader("readme.md")
```

```
88 documents = loader.load()
89
90
91 # Load CSV Files
92
93 from langchain_community.document_loaders.csv_loader import CSVLoader
94
95 loader = CSVLoader("data.csv")
96 documents = loader.load()
97
98 loader = CSVLoader("data.csv")
99 documents = loader.load()
```

44.1 Diff Explanation

— Removed Code

```
- from langchain.document_loaders import PyPDFLoader
- from langchain.document_loaders import Docx2txtLoader
- from langchain.document_loaders import TextLoader
- from langchain.document_loaders import DirectoryLoader
- from langchain.document_loaders import WebBaseLoader
- from langchain.document_loaders import UnstructuredMarkdownLoader
- from langchain.document_loaders.csv_loader import CSVLoader
```

+++ Added Code

```
+ from langchain_community.document_loaders import PyPDFLoader
+ from langchain_community.document_loaders import Docx2txtLoader
+ from langchain_community.document_loaders import TextLoader
+ from langchain_community.document_loaders import DirectoryLoader
+ from langchain_community.document_loaders import WebBaseLoader
+ from langchain_community.document_loaders import UnstructuredMarkdownLoader
+ from langchain_community.document_loaders.csv_loader import CSVLoader
```

44.2 Explanation of the Diff

- **Changed import paths:** From `langchain.document_loaders` to `langchain_community.document_loaders`.
- **Why:** In newer LangChain versions, community-contributed loaders are moved to the `langchain_community` package for better organization.

45 Hugging Face Equivalent Code

```
1  # In a RAG pipeline you must:
2
3  # 1. Collect documents (PDFs, text files, DOCX, website pages, database
   ↪ rows, etc.)
4
5
6  # 2. Load them into your program
7
8
9  # 3. Split them into chunks
10
11
12 # 4. Convert them into embeddings
13
14
15 # 5. Store them in a vector database
16
17
18
19 # Here we focus on Step 2 -- Loading documents.
20
21
22 # ---
23
24 # Most Common Ways to Load Documents for RAG
25
26 # Here are the most frequently used loaders in Python (LangChain):
27
28
29 # ---
30
31 # Load PDFs
32
33 # Using LangChain's PyPDFLoader
34
35 from langchain_community.document_loaders import PyPDFLoader
36
37 loader = PyPDFLoader("sample.pdf")
38 documents = loader.load()
39
40 print(documents)
41
```

```
42
43
44
45 # Load DOCX / Word files
46
47 from langchain_community.document_loaders import Docx2txtLoader
48
49 loader = Docx2txtLoader("sample.docx")
50 documents = loader.load()
51
52
53
54 # Load Text Files (.txt)
55
56 from langchain_community.document_loaders import TextLoader
57
58 loader = TextLoader("notes.txt")
59 documents = loader.load()
60
61
62 # Load Multiple Documents in a Folder
63
64 from langchain_community.document_loaders import DirectoryLoader
65
66 loader = DirectoryLoader(
67     "data/",
68     glob="*/.pdf",
69     show_progress=True
70 )
71
72 documents = loader.load()
73
74
75 # Load Website Pages (HTML)
76
77 from langchain_community.document_loaders import WebBaseLoader
78
79 loader = WebBaseLoader("http://shooliniuniversity.com/")
80 documents = loader.load()
81
82
83 # Load Markdown Files (.md)
84
85 from langchain_community.document_loaders import
86     ↪ UnstructuredMarkdownLoader
87
88 loader = UnstructuredMarkdownLoader("readme.md")
```

```
88 documents = loader.load()
89
90
91 # Load CSV Files
92
93 from langchain_community.document_loaders.csv_loader import CSVLoader
94
95 loader = CSVLoader("data.csv")
96 documents = loader.load()
97
98 loader = CSVLoader("data.csv")
99 documents = loader.load()
```

45.1 Diff Explanation

Same as Ollama: Changed import paths to langchain_community.

45.2 Explanation of the Diff

Same as Ollama.

46 Original Code: code13.py

```
1  from langchain.document_loaders import TextLoader
2
3
4  path='ipl.txt'
5
6  def load_doc(path):
7      if path.endswith('.txt'):
8          loader=TextLoader(path)
9          documents=loader.load()
10         return documents
11
12 documents=load_doc(path)
13 print(documents)
14
15
16 # #Now we do chunking as already told model have limit and answer also
17 ↪ become accurate
18
19 # #1. CharacterTextSplitter
20
21 from langchain.text_splitter import CharacterTextSplitter
22
23 chunking=CharacterTextSplitter(chunk_size=70,chunk_overlap=39)
24
25
26 docs=chunking.split_documents(documents)
27
28 print(len(docs))
29
30
31 for i in range(len(docs)):
32     print(docs[i])
33     print("-----")
```

46.1 Explanation of the Code

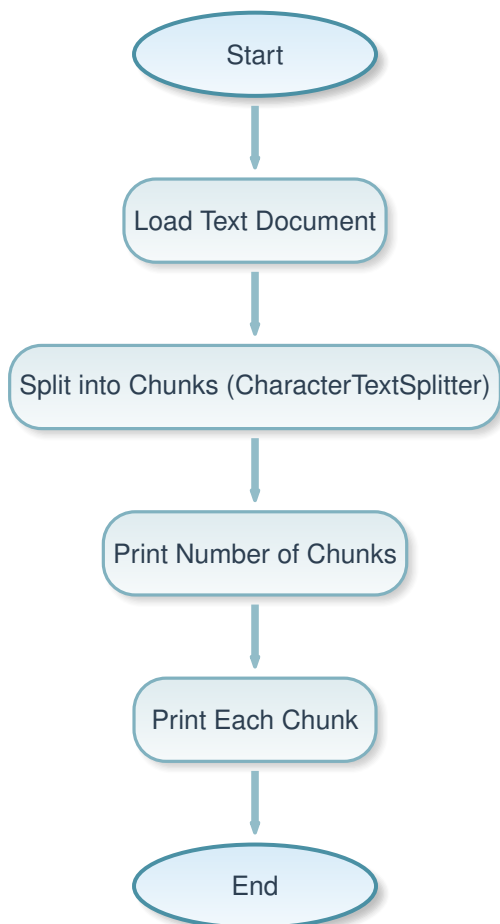
- **What:** This code loads a text file using TextLoader, then splits the document into chunks using CharacterTextSplitter with specified chunk size and overlap, and prints the chunks.
- **Why:** In RAG systems, large documents must be split into manageable chunks to fit model

context limits and improve retrieval accuracy. Character-based splitting is simple and preserves some context via overlap.

- **How:**

- Load text file with TextLoader.
- Create CharacterTextSplitter with chunk_size=70 and chunk_overlap=39.
- Split documents into chunks.
- Print number of chunks and each chunk.

46.2 Flowchart



47 Ollama Equivalent Code

```
1 from langchain_community.document_loaders import TextLoader
2
3
4 path='ipl.txt'
5
6 def load_doc(path):
7     if path.endswith('.txt'):
8         loader=TextLoader(path)
9         documents=loader.load()
10        return documents
11
12 documents=load_doc(path)
13 print(documents)
14
15
16 # #Now we do chunking as already told model have limit and answer also
17 ↪ become accurate
18 # #1. CharacterTextSplitter
19
20 from langchain_text_splitters import CharacterTextSplitter
21
22
23 chunking=CharacterTextSplitter(chunk_size=70,chunk_overlap=39)
24
25
26 docs=chunking.split_documents(documents)
27
28 print(len(docs))
29
30
31 for i in range(len(docs)):
32     print(docs[i])
33     print("-----")
```

47.1 Diff Explanation

— Removed Code

- from langchain.document_loaders import TextLoader
- from langchain.text_splitter import CharacterTextSplitter

+++ Added Code

```
+ from langchain_community.document_loaders import TextLoader
+ from langchain_text_splitters import CharacterTextSplitter
```

47.2 Explanation of the Diff

- **Changed import paths:** Loaders to langchain_community, splitters to langchain_text_splitters.
- **Why:** Modularization in newer LangChain versions.

48 Hugging Face Equivalent Code

Same as Ollama.

48.1 Diff Explanation

Same as Ollama.

49 Original Code: code13A.py

```
1  from langchain.document_loaders import TextLoader
2
3
4  path='ipl.txt'
5
6  def load_doc(path):
7      if path.endswith('.txt'):
8          loader=TextLoader(path)
9          documents=loader.load()
10         return documents
11
12 documents=load_doc(path)
13 print(documents)
14
15
16 # #Now we do chunking as already told model have limit and answer also
17 ↪ become accurate
18
19 import os
20 from langchain_openai import AzureOpenAIEmbeddings
21
22
23 AZURE_OPENAI_API_KEY = "" #Put your own API key
24 AZURE_OPENAI_ENDPOINT = "https://chatgpt-key.openai.azure.com/"
25 AZURE_OPENAI_DEPLOYMENT_NAME = "gpt-4o-2"
26 AZURE_OPENAI_API_VERSION = "2025-01-01-preview"
27 AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME = "text-embedding-3-large" #
28 ↪ Replace with your embedding deployment name
29 AZURE_OPENAI_EMBEDDING_MODEL_NAME = "text-embedding-3-large" # Replace
30 ↪ with your embedding model name
31
32 embeddings = AzureOpenAIEmbeddings(
33     deployment=AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME,
34     model=AZURE_OPENAI_EMBEDDING_MODEL_NAME,
35     api_key=AZURE_OPENAI_API_KEY,
36     azure_endpoint=AZURE_OPENAI_ENDPOINT,
37     api_version=AZURE_OPENAI_API_VERSION,
38 )
39
```

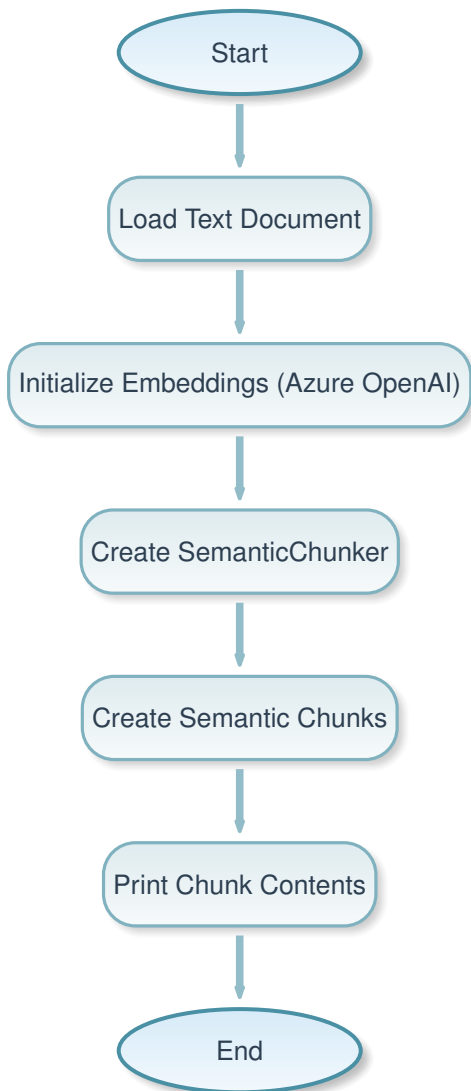


```
40 from langchain_experimental.text_splitter import SemanticChunker
41 from langchain_openai import OpenAIEmbeddings
42
43
44 chunker = SemanticChunker(embeddings)
45 chunks = chunker.create_documents([documents])
46
47 for c in chunks:
48     print(c.page_content)
```

49.1 Explanation of the Code

- **What:** This code loads a text file, initializes Azure OpenAI embeddings, and uses SemanticChunker to split the document into semantically coherent chunks based on embeddings.
- **Why:** Semantic chunking groups text by meaning rather than fixed characters, leading to better retrieval in RAG as chunks maintain topical coherence. Uses embeddings to determine split points.
- **How:**
 - Load text document.
 - Set up Azure OpenAI embeddings with API details.
 - Create SemanticChunker with embeddings.
 - Generate chunks from documents.
 - Print each chunk's content.

49.2 Flowchart



50 Ollama Equivalent Code

```
1 from langchain_community.document_loaders import TextLoader
2
3
4 path='ipl.txt'
5
6 def load_doc(path):
7     if path.endswith('.txt'):
```

```

8     loader=TextLoader(path)
9     documents=loader.load()
10    return documents
11
12    documents=load_doc(path)
13    print(documents)
14
15
16    # Now we do chunking as already told model have limit and answer also
17    ↔ become accurate
18
19    from langchain_community.embeddings import HuggingFaceEmbeddings
20
21
22    embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
23
24
25    from langchain_experimental.text_splitter import SemanticChunker
26
27
28    chunker = SemanticChunker(embeddings)
29    chunks = chunker.create_documents([documents])
30
31    for c in chunks:
32        print(c.page_content)

```

50.1 Diff Explanation

— Removed Code

```

- from langchain.document_loaders import TextLoader
- import os
- from langchain_openai import AzureOpenAIEmbeddings
- AZURE_OPENAI_API_KEY = "" ... (Azure setup)
- embeddings = AzureOpenAIEmbeddings(...)
- from langchain_experimental.text_splitter import SemanticChunker
- from langchain_openai import OpenAIEmbeddings

```

+++ Added Code

```

+ from langchain_community.document_loaders import TextLoader
+ from langchain_community.embeddings import HuggingFaceEmbeddings
+ embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")

```

```
+ from langchain_experimental.text_splitter import SemanticChunker
```

50.2 Explanation of the Diff

- **Removed Azure embeddings:** Replaced with local HuggingFace embeddings.
- **Changed imports:** To langchain_community.
- **Why:** Adapts to local embeddings for Ollama, avoiding cloud API costs.

51 Hugging Face Equivalent Code

Same as Ollama.

51.1 Diff Explanation

Same as Ollama.

52 Original Code: code14.py

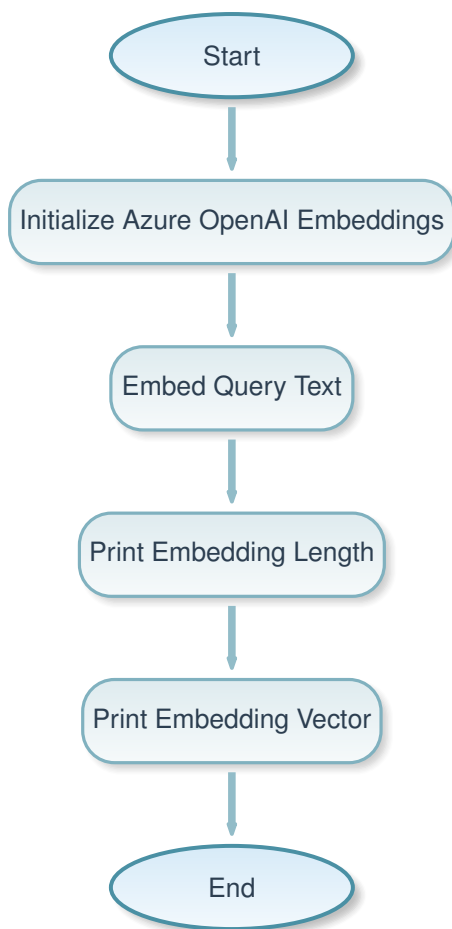
```
1  import os
2  from langchain_openai import AzureOpenAIEmbeddings
3
4  AZURE_OPENAI_API_KEY = "" #Put your own API key
5  AZURE_OPENAI_ENDPOINT = "https://chatgpt-key.openai.azure.com/"
6  AZURE_OPENAI_DEPLOYMENT_NAME = "gpt-4o-2"
7  AZURE_OPENAI_API_VERSION = "2025-01-01-preview"
8  AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME = "text-embedding-3-large" #
   ↪ Replace with your embedding deployment name
9  AZURE_OPENAI_EMBEDDING_MODEL_NAME = "text-embedding-3-large" # Replace
   ↪ with your embedding model name
10
11
12  embeddings = AzureOpenAIEmbeddings(
13      deployment=AZURE_OPENAI_EMBEDDING_DEPLOYMENT_NAME,
14      model=AZURE_OPENAI_EMBEDDING_MODEL_NAME,
15      api_key=AZURE_OPENAI_API_KEY,
16      azure_endpoint=AZURE_OPENAI_ENDPOINT,
17      api_version=AZURE_OPENAI_API_VERSION,
18  )
19
20  text = "I love machine learning"
21
22
23  embedding = embeddings.embed_query(text)
24
25  print(f"Embedding vector length: {len(embedding)}")
26  print(embedding)
```

52.1 Explanation of the Code

- **What:** This code initializes Azure OpenAI embeddings and generates an embedding vector for the text "I love machine learning", then prints the vector length and the vector.
- **Why:** Embeddings convert text into numerical vectors for semantic similarity in RAG systems. Azure provides high-quality embeddings via API.
- **How:**
 - Set up Azure OpenAI embeddings with API details.

- Call `embed_query` on the text.
- Print length and vector.

52.2 Flowchart



53 Ollama Equivalent Code

```
1 from langchain_community.embeddings import HuggingFaceEmbeddings
2
3 embedding_model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
4
5 text = "I love machine learning"
6
7 embedding = embedding_model.embed_query(text)
8
```

```
9 print(f"Embedding vector length: {len(embedding)}")
10 print(embedding)
```

53.1 Diff Explanation

— Removed Code

```
- import os
- from langchain_openai import AzureOpenAIEmbeddings
- AZURE_OPENAI_API_KEY = "" ... (Azure setup)
- embeddings = AzureOpenAIEmbeddings(...)
```

+++ Added Code

```
+ from langchain_community.embeddings import HuggingFaceEmbeddings
+ embedding_model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
```

53.2 Explanation of the Diff

- **Removed Azure setup:** Local embeddings.
- **Changed to HuggingFace:** Uses BAAI/bge-m3 model.
- **Why:** Local execution without API keys.

54 Hugging Face Equivalent Code

```
1 from FlagEmbedding import BGEM3FlagModel
2
3 model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)
4
5 sentence1="I love machine learning"
6
7 embeddings1=model.encode(sentence1,
8                           batch_size=12,
9                           max_length=8192,)['dense_vecs']
10
11 print(embeddings1)
```

54.1 Diff Explanation

— Removed Code

```
- from langchain_community.embeddings import HuggingFaceEmbeddings
- embedding_model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
- embedding = embedding_model.embed_query(text)
- print(f"Embedding vector length: {len(embedding)}")
- print(embedding)
```

+++ Added Code

```
+ from FlagEmbedding import BGEM3FlagModel
+ model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)
+ sentence1="I love machine learning"
+ embeddings1=model.encode(sentence1, batch_size=12, max_length=8192,)['dense_vecs']
+ print(embeddings1)
```

54.2 Explanation of the Diff

- **Removed LangChain wrapper:** Direct FlagEmbedding usage.
- **Changed method:** encode instead of embed_query.
- **Why:** Direct access to Hugging Face embedding models for more control.

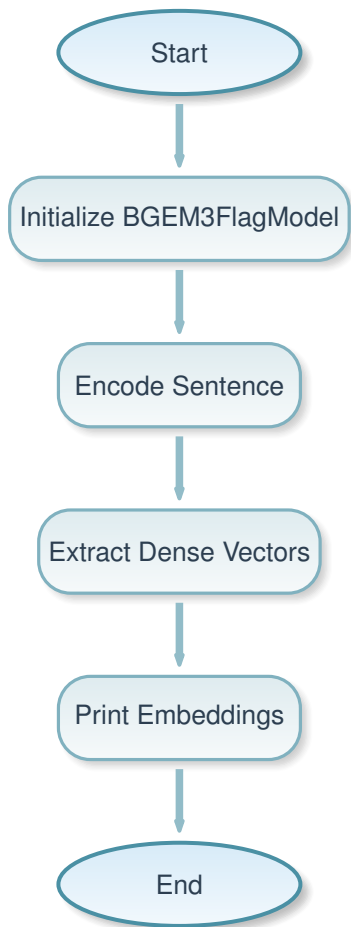
55 Original Code: code14A.py

```
1  # !pip install -U FlagEmbedding
2
3  from FlagEmbedding import BGEM3FlagModel
4
5  model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)
6
7  sentence1="I am going to school"
8
9  embeddings1=model.encode(sentence1,
10                           batch_size=12,
11                           max_length=8192,)['dense_vecs']
12
13  print(embeddings1)
```

55.1 Explanation of the Code

- **What:** This code loads the BGE-M3 embedding model from FlagEmbedding and generates dense embeddings for the sentence "I am going to school", then prints the embedding vector.
- **Why:** Direct use of advanced embedding models for text representation. BGE-M3 is a state-of-the-art model for semantic embeddings.
- **How:**
 - Install FlagEmbedding if needed.
 - Initialize BGEM3FlagModel with model name and fp16.
 - Encode the sentence with parameters.
 - Extract and print dense vectors.

55.2 Flowchart



56 Ollama Equivalent Code

Same as original.

56.1 Diff Explanation

No changes.

56.2 Explanation of the Diff

All versions use the same direct FlagEmbedding approach.

57 Hugging Face Equivalent Code

Same as original.

57.1 Diff Explanation

No changes.

58 Original Code (src/code15.py)

The original code demonstrates computing cosine similarity between two sentences using the BGEM3FlagModel from FlagEmbedding.

```
1  #cosine similarity
2
3
4  # !pip install -U FlagEmbedding
5
6  from FlagEmbedding import BGEM3FlagModel
7  from sklearn.metrics.pairwise import cosine_similarity
8
9  model=BGEM3FlagModel('BAAI/bge-m3',use_fp16=True)
10
11 sentence1="I am going to school"
12
13 embeddings1=model.encode(sentence1,
14                             batch_size=12,
15                             max_length=8192,)['dense_vecs']
16
17 print(embeddings1)
18
19 sentence2="I am going to home"
20
21 embeddings2=model.encode(sentence2,
22                             batch_size=12,
23                             max_length=8192,)['dense_vecs']
24
25 print(embeddings2)
26
27 similarity_list= cosine_similarity(
28     [embeddings1],
29     [embeddings2]
30 ) [0] [0]
31
32 print(similarity)
```

59 Explanation

59.1 What

This code computes the cosine similarity between the embeddings of two sentences. Cosine similarity measures the cosine of the angle between two vectors in a multi-dimensional space, providing a value between -1 and 1 that indicates how similar the vectors (and thus the sentences) are semantically.

59.2 Why

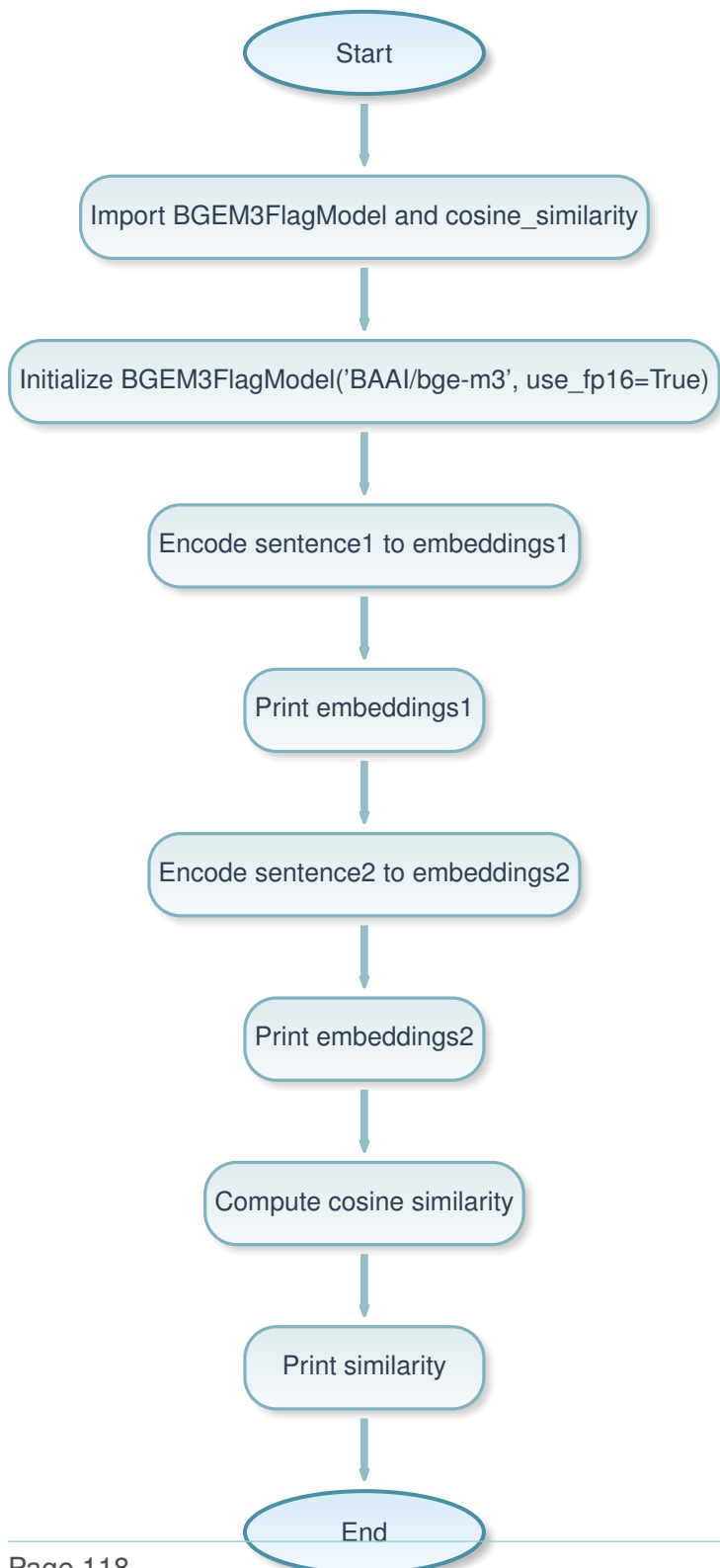
Sentence embeddings represent the semantic meaning of text in a numerical vector form. By comparing these vectors using cosine similarity, we can quantify how closely related two pieces of text are. This is useful in applications like semantic search, text clustering, and recommendation systems where understanding textual similarity is key.

59.3 How

1. Import the BGEM3FlagModel from FlagEmbedding and cosine_similarity from sklearn.
2. Initialize the model with the 'BAAI/bge-m3' pre-trained model, enabling FP16 for efficiency.
3. Encode two sentences into dense vector embeddings using the model's encode method, specifying batch size and max length.
4. Compute the cosine similarity between the two embedding vectors.
5. Print the embeddings and the similarity score.

Note: There is a typo in the code: 'similarity_list' should be 'similarity', and 'print(similarity)' at the end.

60 Flowchart



61 Original Code (src/code15A.py)

The original code demonstrates computing cosine similarity between embeddings of a text document and a query using Azure OpenAI Embeddings.

```

1  from langchain_openai import AzureOpenAIEmbeddings
2  import os
3
4  embedding_model = AzureOpenAIEmbeddings(
5      openai_api_key="",
6      azure_deployment="text-embedding-3-large",
7      azure_endpoint="https://chatgpt-key.openai.azure.com",
8      openai_api_type="azure",
9      openai_api_version="2023-05-15",
10     chunk_size=1000
11 )
12
13
14 text = '''
15 Marwadi University (MU)[1] is a private university[2] located in Rajkot,
    ↳ Gujarat, India. It was established on 9 May 2016 by the Marwadi
    ↳ Education Foundation through The Gujarat Private Universities (
    ↳ Amendment) Act, 2016.[3] As of 2017, it offers 54 different courses
    ↳ .[4] It is graded A+ by NAAC.[5]
16
17 The university operates under the division of Marwadi Education Foundation
    ↳ 's Group of Institutions (MEFGI). MEFGI commenced its operations in
    ↳ the year 2008. It was established as a primary unit of Marwadi
    ↳ Education Foundation under the Bombay Public Trust Act 1950. Marwadi
    ↳ University is aided by the Marwadi Shares and Finance Limited, a
    ↳ stock broking company in India and Chandarana Intermediaries Brokers
    ↳ Pvt. Ltd. (CIBPL), a firm dealing in technical and arbitrage
    ↳ trading.[6]
18
19 Campus
20 The campus is located on 52 acres of land, having a distance of nearly 40
    ↳ minutes from railway and airports. The university comprises eight
    ↳ multi-storey buildings.[7] Laboratories, research facilities,
    ↳ student clubs, sports club and college cafeteria are available.[8]
21
22 There are two libraries with RFID technologies, 60+ computer systems,
    ↳ 50000+ books.[9] The campus also includes banking and ATM facilities
    ↳ .[10] Around 70+ buses function every day at regular intervals for
    ↳ students and staff.[11] There are hostel rooms with internet

```

```

    ↪ facilities, laundry, dance rooms, libraries etc. and capacity to
    ↪ occupy over 2000 students.[12]
23
24     '''
25
26
27     query="What is shoolini University?"
28
29
30
31
32     embedding_text = embedding_model.embed_query(text)
33
34     embedding_query = embedding_model.embed_query(query)
35
36
37     from sklearn.metrics.pairwise import cosine_similarity
38
39
40
41
42     sim = cosine_similarity(
43         [embeddings_query],    # query embedding
44         [embedding_text]       # document embedding
45     )[0][0]
46
47
48
49     print(f"Embedding vector length: {len(embedding)}")
50     print(embedding_text)
51     print
52     ↪ ("-----")
53     ↪
54     print(f"Embedding vector length: {len(embedding_query)}")
55     print(sim)

```

62 Explanation

62.1 What

This code computes the cosine similarity between the embeddings of a long text document (about Marwadi University) and a query string using Azure OpenAI's text-embedding-3-large

model. It demonstrates how to use cloud-based embeddings for semantic similarity measurement.

62.2 Why

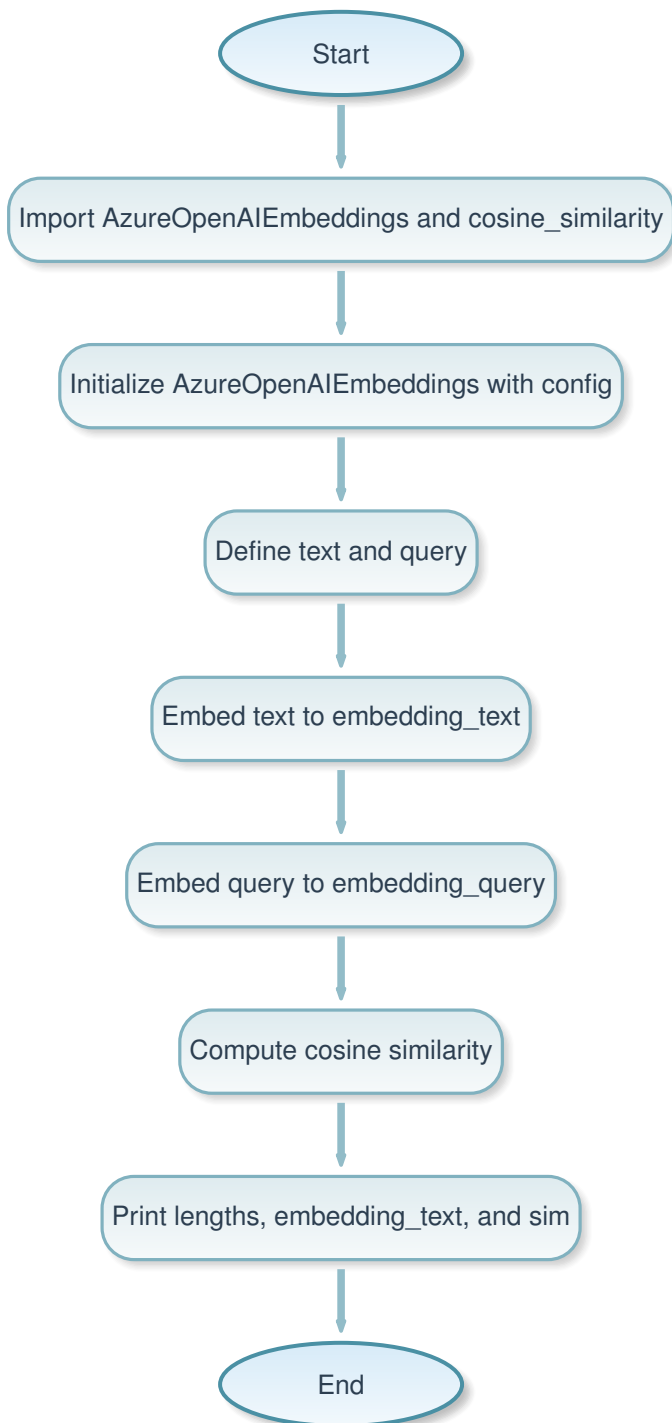
Embeddings from large language models like OpenAI's can capture rich semantic information. Cosine similarity on these embeddings allows quantifying how relevant a document is to a query, which is fundamental for information retrieval, question-answering systems, and RAG (Retrieval-Augmented Generation) pipelines.

62.3 How

1. Import `AzureOpenAIEmbeddings` from `langchain_openai` and `cosine_similarity` from `sklearn`. 2. Initialize the embedding model with Azure OpenAI credentials and configuration (deployment, endpoint, API version, chunk size). 3. Define a long text document and a query. 4. Embed both the text and query using the model's `embed_query` method. 5. Compute cosine similarity between the query and text embeddings. 6. Print the embedding vector lengths, the text embedding, and the similarity score.

Note: There are typos in the code: `'embeddings_query'` should be `'embedding_query'`, and `'len(embedding)'` should be `'len(embedding_text)'`.

63 Flowchart



64 Ollama Equivalent Code

```

1  from langchain_community.embeddings import HuggingFaceEmbeddings
2  from sklearn.metrics.pairwise import cosine_similarity
3
4  embedding_model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')
5
6  text = '''
7  Marwadi University (MU)[1] is a private university[2] located in Rajkot,
    ↳ Gujarat, India. It was established on 9 May 2016 by the Marwadi
    ↳ Education Foundation through The Gujarat Private Universities (
    ↳ Amendment) Act, 2016.[3] As of 2017, it offers 54 different courses
    ↳ .[4] It is graded A+ by NAAC.[5]
8
9  The university operates under the division of Marwadi Education Foundation
    ↳ 's Group of Institutions (MEFGI). MEFGI commenced its operations in
    ↳ the year 2008. It was established as a primary unit of Marwadi
    ↳ Education Foundation under the Bombay Public Trust Act 1950. Marwadi
    ↳ University is aided by the Marwadi Shares and Finance Limited, a
    ↳ stock broking company in India and Chandarana Intermediaries Brokers
    ↳ Pvt. Ltd. (CIBPL), a firm dealing in technical and arbitrage
    ↳ trading.[6]
10
11  Campus
12  The campus is located on 52 acres of land, having a distance of nearly 40
    ↳ minutes from railway and airports. The university comprises eight
    ↳ multi-storey buildings.[7] Laboratories, research facilities,
    ↳ student clubs, sports club and college cafeteria are available.[8]
13
14  There are two libraries with RFID technologies, 60+ computer systems,
    ↳ 50000+ books.[9] The campus also includes banking and ATM facilities
    ↳ .[10] Around 70+ buses function every day at regular intervals for
    ↳ students and staff.[11] There are hostel rooms with internet
    ↳ facilities, laundry, dance rooms, libraries etc. and capacity to
    ↳ occupy over 2000 students.[12]
15
16  '''
17
18  query="What is shoolini University?"
19
20  embedding_text = embedding_model.embed_query(text)
21
22  embedding_query = embedding_model.embed_query(query)
23

```

```

24  sim = cosine_similarity(
25      [embedding_query],    # query embedding
26      [embedding_text]      # document embedding
27  )[0][0]
28
29  print(f"Embedding vector length: {len(embedding_text)}")
30  print(embedding_text)
31  print
32      ↪ ("-----")
33      ↪
32  print(f"Embedding vector length: {len(embedding_query)}")
33  print(sim)

```

64.1 Diff Explanation

The Ollama equivalent uses LangChain's HuggingFaceEmbeddings with a local BGE-M3 model instead of Azure OpenAI, and fixes the variable name typos.

Key changes:

- **Removed:** `from langchain_openai import AzureOpenAIEmbeddings`
- **Added:** `from langchain_community.embeddings import HuggingFaceEmbeddings`
- **Removed:** AzureOpenAIEmbeddings initialization with API keys
- **Added:** `embedding_model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')`
- **Fixed:** `[embedding_query]` instead of `[embeddings_query]`
- **Fixed:** `len(embedding_text)` instead of `len(embedding)`

These changes replace cloud-based embeddings with local, open-source ones for privacy and cost reasons, while maintaining the same similarity computation logic.

65 Hugging Face Equivalent Code

```

1  from FlagEmbedding import BGEM3FlagModel
2  from sklearn.metrics.pairwise import cosine_similarity
3
4  embedding_model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)
5

```

```

6  text = '''
7  Marwadi University (MU)[1] is a private university[2] located in Rajkot,
    ↳ Gujarat, India. It was established on 9 May 2016 by the Marwadi
    ↳ Education Foundation through The Gujarat Private Universities (
    ↳ Amendment) Act, 2016.[3] As of 2017, it offers 54 different courses
    ↳ .[4] It is graded A+ by NAAC.[5]
8
9  The university operates under the division of Marwadi Education Foundation
    ↳ 's Group of Institutions (MEFGI). MEFGI commenced its operations in
    ↳ the year 2008. It was established as a primary unit of Marwadi
    ↳ Education Foundation under the Bombay Public Trust Act 1950. Marwadi
    ↳ University is aided by the Marwadi Shares and Finance Limited, a
    ↳ stock broking company in India and Chandarana Intermediaries Brokers
    ↳ Pvt. Ltd. (CIBPL), a firm dealing in technical and arbitrage
    ↳ trading.[6]
10
11  Campus
12  The campus is located on 52 acres of land, having a distance of nearly 40
    ↳ minutes from railway and airports. The university comprises eight
    ↳ multi-storey buildings.[7] Laboratories, research facilities,
    ↳ student clubs, sports club and college cafeteria are available.[8]
13
14  There are two libraries with RFID technologies, 60+ computer systems,
    ↳ 50000+ books.[9] The campus also includes banking and ATM facilities
    ↳ .[10] Around 70+ buses function every day at regular intervals for
    ↳ students and staff.[11] There are hostel rooms with internet
    ↳ facilities, laundry, dance rooms, libraries etc. and capacity to
    ↳ occupy over 2000 students.[12]
15
16  '''
17
18  query="What is shoolini University?"
19
20  embedding_text = embedding_model.encode(text, batch_size=12, max_length
    ↳ =8192)['dense_vecs']
21
22  embedding_query = embedding_model.encode(query, batch_size=12, max_length
    ↳ =8192)['dense_vecs']
23
24  sim = cosine_similarity(
25      [embedding_query],    # query embedding
26      [embedding_text]      # document embedding
27  )[0][0]
28
29  print(f"Embedding vector length: {len(embedding_text)}")
30  print(embedding_text)
31  print

```

```
↔ ("-----")  
↔  
32 print(f"Embedding vector length: {len(embedding_query)}")  
33 print(sim)
```

65.1 Diff Explanation

The Hugging Face equivalent uses the direct BGEM3FlagModel from FlagEmbedding instead of LangChain wrappers, and uses the encode method with parameters.

Key changes:

- **Removed:** LangChain HuggingFaceEmbeddings
- **Added:** `from FlagEmbedding import BGEM3FlagModel`
- **Removed:** `embedding_model = HuggingFaceEmbeddings(...)`
- **Added:** `embedding_model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)`
- **Removed:** `embed_query` method
- **Added:** `encode` method with `batch_size` and `max_length` parameters
- **Fixed:** Variable names consistent

These changes use the native FlagEmbedding library for more direct control over encoding parameters, suitable for high-performance local inference.

66 Original Code (src/code16.py)

The original code implements a movie recommendation system based on semantic similarity between user-input genres and movie descriptions using BGEM3FlagModel embeddings.

```
1  # !pip install -U FlagEmbedding
2
3  from FlagEmbedding import BGEM3FlagModel
4  from sklearn.metrics.pairwise import cosine_similarity
5
6  model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)
7
8
9  movies_list = [
10     "Shadow Hunters: ACTION FANTASY THRILLER",
11     "Eternal Bonds: DRAMA ROMANCE FAMILY",
12     "Neon Horizon: SCI-FI ACTION ADVENTURE",
13     "Whispers in the Dark: HORROR MYSTERY THRILLER",
14     "Crimson Crown: HISTORICAL WAR DRAMA",
15     "Starlight Dreams: ROMANCE COMEDY MUSIC",
16     "Code Breakers: TECH THRILLER CRIME",
17     "The Lost Kingdom: ADVENTURE ACTION FANTASY",
18     "Broken Silence: CRIME DRAMA SUSPENSE",
19     "Frozen Ashes: SURVIVAL DRAMA ACTION"
20 ]
21
22 # Convert movie list into embeddings
23 embeddings_list = []
24 for movie in movies_list:
25     emb = model.encode(
26         movie,
27         batch_size=12,
28         max_length=8192
29     )['dense_vecs']
30     embeddings_list.append(emb)
31
32
33 query = input("ENTER THE GENRE FOR WHICH YOU WANT TO SEE MOVIES: ")
34
35
36 query_embedding = model.encode(
37     query,
38     batch_size=12,
39     max_length=8192
```

```
40 )['dense_vecs']
41
42
43 similarity_scores = {}
44
45 for i, movie_emb in enumerate(embeddings_list):
46     similarity = cosine_similarity(
47         [query_embedding],
48         [movie_emb]
49     )[0][0]
50
51     similarity_scores[movies_list[i]] = similarity
52
53
54 sorted_movies = sorted(similarity_scores.items(), key=lambda x: x[1],
55                        ↪ reverse=True)
56
57 #Retrived data
58 print("\n Top Recommended Movies:")
59 for movie, score in sorted_movies[:4]:
60     print(f"{movie}    --> Similarity: {score:.4f}")
```

67 Explanation

67.1 What

This code builds a simple movie recommendation system by computing semantic similarity between a user-provided genre query and a list of movie descriptions (including genres). It uses sentence embeddings to find movies whose descriptions are most similar to the query.

67.2 Why

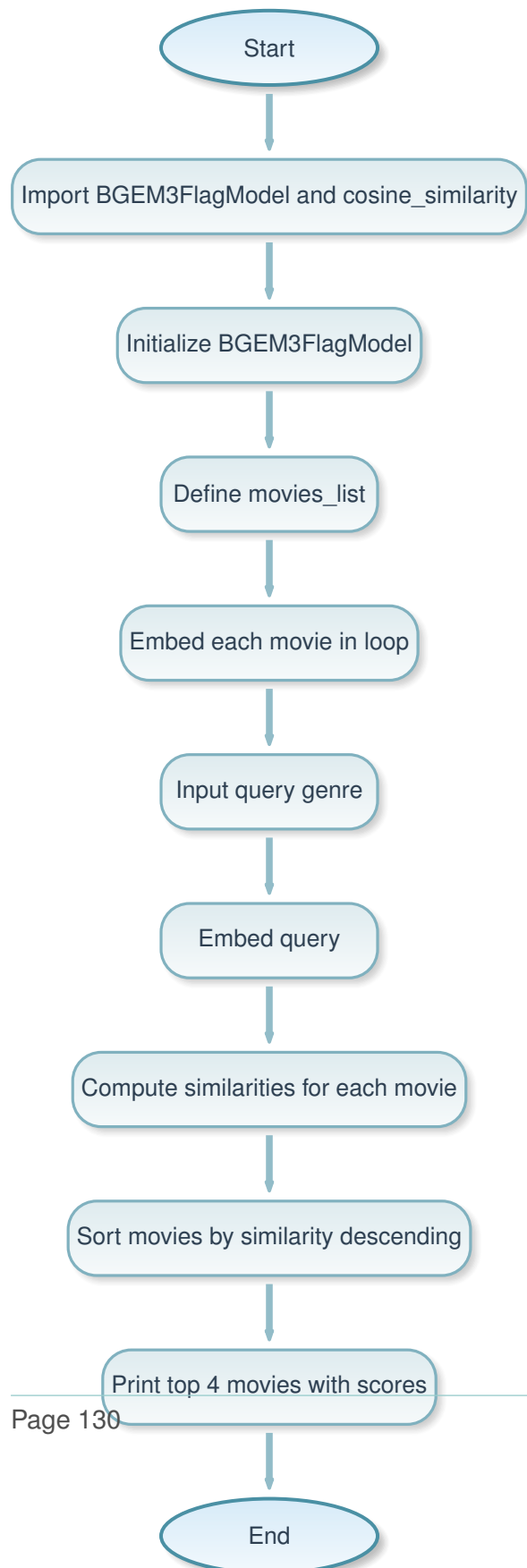
Traditional keyword-based search might miss movies with related but not exact genre matches (e.g., "action" vs. "adventure action"). Semantic embeddings capture meaning, allowing recommendations based on conceptual similarity, improving user experience in content discovery.

67.3 How

1. Import BGEM3FlagModel and cosine_similarity.
2. Initialize the embedding model.
3. Define a list of movie descriptions with genres.
4. Embed each movie description into vectors

and store in a list. 5. Take user input for a genre query. 6. Embed the query into a vector. 7. Compute cosine similarity between the query embedding and each movie embedding. 8. Sort movies by similarity score descending. 9. Print the top 4 recommended movies with their scores.

68 Flowchart



69 Ollama Equivalent Code

```

1  from langchain_community.embeddings import HuggingFaceEmbeddings
2  from sklearn.metrics.pairwise import cosine_similarity
3
4  model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')
5
6  movies_list = [
7      "Shadow Hunters: ACTION FANTASY THRILLER",
8      "Eternal Bonds: DRAMA ROMANCE FAMILY",
9      "Neon Horizon: SCI-FI ACTION ADVENTURE",
10     "Whispers in the Dark: HORROR MYSTERY THRILLER",
11     "Crimson Crown: HISTORICAL WAR DRAMA",
12     "Starlight Dreams: ROMANCE COMEDY MUSIC",
13     "Code Breakers: TECH THRILLER CRIME",
14     "The Lost Kingdom: ADVENTURE ACTION FANTASY",
15     "Broken Silence: CRIME DRAMA SUSPENSE",
16     "Frozen Ashes: SURVIVAL DRAMA ACTION"
17 ]
18
19 # Convert movie list into embeddings
20 embeddings_list = model.embed_documents(movies_list)
21
22 query = input("ENTER THE GENRE FOR WHICH YOU WANT TO SEE MOVIES: ")
23
24 query_embedding = model.embed_query(query)
25
26 similarity_scores = {}
27
28 for i, movie_emb in enumerate(embeddings_list):
29     similarity = cosine_similarity(
30         [query_embedding],
31         [movie_emb]
32     )[0][0]
33
34     similarity_scores[movies_list[i]] = similarity
35
36 sorted_movies = sorted(similarity_scores.items(), key=lambda x: x[1],
37     ↪ reverse=True)
38
39 #Retrieved data
40 print("\n Top Recommended Movies:")
41 for movie, score in sorted_movies[:4]:
42     print(f"{movie}    --> Similarity: {score:.4f}")

```

69.1 Diff Explanation

The Ollama equivalent uses LangChain's HuggingFaceEmbeddings wrapper, which simplifies embedding multiple documents and queries with dedicated methods.

Key changes:

- **Removed:** `from FlagEmbedding import BGEM3FlagModel`
- **Added:** `from langchain_community.embeddings import HuggingFaceEmbeddings`
- **Removed:** `model = BGEM3FlagModel(...)`
- **Added:** `model = HuggingFaceEmbeddings(model_name='BAAI/bge-m3')`
- **Removed:** Loop to encode each movie
- **Added:** `embeddings_list = model.embed_documents(movies_list)`
- **Removed:** `model.encode(query)['dense_vecs']`
- **Added:** `query_embedding = model.embed_query(query)`

These changes leverage LangChain's abstraction for batch document embedding and query embedding, making the code more concise and integrated with LangChain ecosystems, while achieving the same recommendation logic.

70 Hugging Face Equivalent Code

```
1 from FlagEmbedding import BGEM3FlagModel
2 from sklearn.metrics.pairwise import cosine_similarity
3
4 model = BGEM3FlagModel('BAAI/bge-m3', use_fp16=True)
5
6 movies_list = [
7     "Shadow Hunters: ACTION FANTASY THRILLER",
8     "Eternal Bonds: DRAMA ROMANCE FAMILY",
9     "Neon Horizon: SCI-FI ACTION ADVENTURE",
10    "Whispers in the Dark: HORROR MYSTERY THRILLER",
11    "Crimson Crown: HISTORICAL WAR DRAMA",
12    "Starlight Dreams: ROMANCE COMEDY MUSIC",
13    "Code Breakers: TECH THRILLER CRIME",
14    "The Lost Kingdom: ADVENTURE ACTION FANTASY",
15    "Broken Silence: CRIME DRAMA SUSPENSE",
```

```

16     "Frozen Ashes: SURVIVAL DRAMA ACTION"
17 ]
18
19 # Convert movie list into embeddings
20 embeddings_list = []
21 for movie in movies_list:
22     emb = model.encode(
23         movie,
24         batch_size=12,
25         max_length=8192
26     )['dense_vecs']
27     embeddings_list.append(emb)
28
29 query = input("ENTER THE GENRE FOR WHICH YOU WANT TO SEE MOVIES: ")
30
31 query_embedding = model.encode(
32     query,
33     batch_size=12,
34     max_length=8192
35 )['dense_vecs']
36
37 similarity_scores = {}
38
39 for i, movie_emb in enumerate(embeddings_list):
40     similarity = cosine_similarity(
41         [query_embedding],
42         [movie_emb]
43     )[0][0]
44
45     similarity_scores[movies_list[i]] = similarity
46
47 sorted_movies = sorted(similarity_scores.items(), key=lambda x: x[1],
48                        ↪ reverse=True)
49
50 #Retrieved data
51 print("\n Top Recommended Movies:")
52 for movie, score in sorted_movies[:4]:
53     print(f"{movie}    --> Similarity: {score:.4f}")

```

70.1 Diff Explanation

The Hugging Face equivalent is nearly identical to the original, using the same BGEM3FlagModel and encoding approach.

Key changes: None significant; the code is essentially the same, with minor formatting differ-

ences (e.g., comment "Retrieved data" instead of "Retrived data").

This indicates that for direct Hugging Face usage, the original code is already optimal, requiring no adaptations.

71 Original Code (src/code17.py)

The original code implements a complete Retrieval-Augmented Generation (RAG) pipeline for movie recommendations, using embeddings for retrieval and an LLM for generation.

```

1  # =====
2  # RAG PIPELINE WITH EXTERNAL MOVIE DATASET -- MODULAR VERSION
3  # =====
4
5  # !pip install -U FlagEmbedding langchain-openai python-dotenv
6
7  from FlagEmbedding import BGEM3FlagModel
8  from sklearn.metrics.pairwise import cosine_similarity
9  from langchain.chat_models import AzureChatOpenAI
10 from dotenv import load_dotenv
11 import os
12
13 load_dotenv()
14
15 # -----
16 # FUNCTION 1: Load Movie Dataset
17 # -----
18 def load_movie_dataset():
19     return [
20         "Shadow Hunters: ACTION FANTASY THRILLER",
21         "Eternal Bonds: DRAMA ROMANCE FAMILY",
22         "Neon Horizon: SCI-FI ACTION ADVENTURE",
23         "Whispers in the Dark: HORROR MYSTERY THRILLER",
24         "Crimson Crown: HISTORICAL WAR DRAMA",
25         "Starlight Dreams: ROMANCE COMEDY MUSIC",
26         "Code Breakers: TECH THRILLER CRIME",
27         "The Lost Kingdom: ADVENTURE ACTION FANTASY",
28         "Broken Silence: CRIME DRAMA SUSPENSE",
29         "Frozen Ashes: SURVIVAL DRAMA ACTION"
30     ]
31
32 # -----
33 # FUNCTION 2: Load Embedding Model + Generate Embeddings
34 # -----
35 def embed_movies(movies_list):
36     model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True)
37     embeddings = [
38         model.encode(movie, batch_size=12, max_length=8192)["dense_vecs"]
39         for movie in movies_list

```

```

40     ]
41     return model, embeddings
42
43     # -----
44     # FUNCTION 3: Retrieve Top-K Relevant Movies
45     # -----
46     def retrieve_top_movies(query, model, movies_list, movie_embeddings, top_k
47         ↪ =4):
48         query_emb = model.encode(query, batch_size=12, max_length=8192) ["
49         ↪ dense_vecs"]
48
49         similarity_scores = {}
50         for i, emb in enumerate(movie_embeddings):
51             sim = cosine_similarity([query_emb], [emb])[0][0]
52             similarity_scores[movies_list[i]] = sim
53
54         sorted_results = sorted(
55             similarity_scores.items(), key=lambda x: x[1], reverse=True
56         )
57         return sorted_results[:top_k]
58
59     # -----
60     # FUNCTION 4: Build the RAG Prompt
61     # -----
62     def build_rag_prompt(query, retrieved_movies):
63         context_text = "\n".join([f"- {m[0]}" for m in retrieved_movies])
64
65         prompt = f"""
66         You are a movie recommendation expert.
67
68         USER QUERY:
69         {query}
70
71         RETRIEVED MOVIES (Context Provided):
72         {context_text}
73
74         TASK:
75         Using ONLY the above movies as your context, recommend movies to the user.
76         Clearly explain why each recommended movie matches the user's taste.
77         Do NOT mention similarity scores or embeddings.
78         """
79         return prompt
80
81     # -----
82     # FUNCTION 5: Invoke Azure OpenAI LLM
83     # -----
84     def generate_llm_response(prompt):

```



```
85     llm = AzureChatOpenAI(
86         openai_api_base=os.getenv("AZURE_OPENAI_API_BASE"),
87         openai_api_version=os.getenv("AZURE_OPENAI_API_VERSION"),
88         openai_api_key=os.getenv("AZURE_OPENAI_API_KEY"),
89         deployment_name=os.getenv("AZURE_OPENAI_DEPLOYMENT_NAME"),
90         model_name="gpt-4o",
91         temperature=0.7,
92     )
93
94     response = llm.invoke(prompt)
95     return response.content
96
97 # -----
98 # FUNCTION 6: Full RAG Pipeline Executor
99 # -----
100 def run_rag_pipeline():
101     # Load dataset
102     movies_list = load_movie_dataset()
103
104     # Embeddings
105     model, movie_embeddings = embed_movies(movies_list)
106
107     # User Input
108     query = input("Enter movie genre or mood: ")
109
110     # Retrieve
111     retrieved = retrieve_top_movies(query, model, movies_list,
112     ↪ movie_embeddings)
113
114     # Build prompt
115     prompt = build_rag_prompt(query, retrieved)
116
117     # LLM response
118     answer = generate_llm_response(prompt)
119
120     print("\n===== RAG ANSWER =====")
121     print(answer)
122
123 # -----
124 # RUN PIPELINE
125 # -----
126 run_rag_pipeline()
```

72 Explanation

72.1 What

This code builds a modular Retrieval-Augmented Generation (RAG) pipeline for movie recommendations. It retrieves relevant movies from a dataset based on semantic similarity to a user query, then uses an LLM to generate personalized recommendations with explanations.

72.2 Why

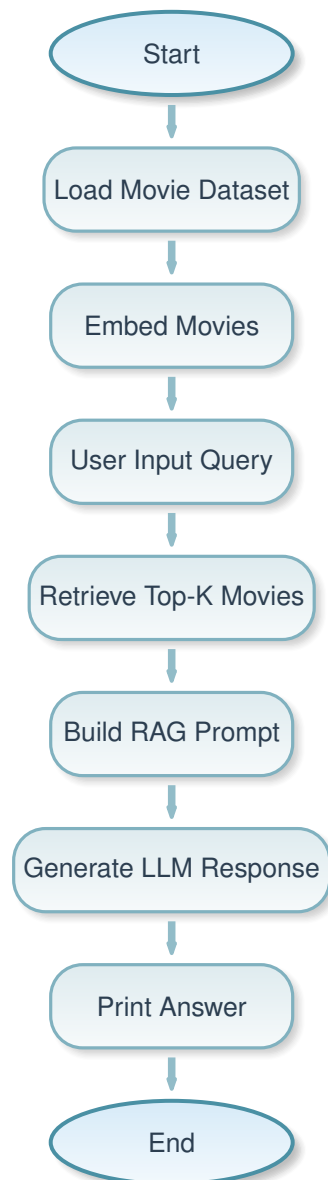
RAG combines the strengths of retrieval (finding relevant information) and generation (creating coherent responses). This prevents hallucinations in LLMs by grounding responses in retrieved data, making recommendations more accurate and contextually relevant.

72.3 How

1. Load a dataset of movie descriptions. 2. Embed all movies using BGEM3FlagModel. 3. Take user input for genre/mood. 4. Embed the query and retrieve top-K similar movies via cosine similarity. 5. Build a prompt with the query and retrieved movies as context. 6. Invoke Azure OpenAI GPT-4o to generate a recommendation response. 7. Print the final answer.

The modular structure separates concerns: data loading, embedding, retrieval, prompt building, LLM invocation, and pipeline execution.

73 Flowchart



74 Ollama Equivalent Code

```
1 # =====  
2 # RAG PIPELINE WITH EXTERNAL MOVIE DATASET -- MODULAR VERSION
```

```
3  # =====
4
5  from langchain_community.embeddings import HuggingFaceEmbeddings
6  from sklearn.metrics.pairwise import cosine_similarity
7  from langchain_ollama import OllamaLLM
8
9  # -----
10 # FUNCTION 1: Load Movie Dataset
11 # -----
12 def load_movie_dataset():
13     return [
14         "Shadow Hunters: ACTION FANTASY THRILLER",
15         "Eternal Bonds: DRAMA ROMANCE FAMILY",
16         "Neon Horizon: SCI-FI ACTION ADVENTURE",
17         "Whispers in the Dark: HORROR MYSTERY THRILLER",
18         "Crimson Crown: HISTORICAL WAR DRAMA",
19         "Starlight Dreams: ROMANCE COMEDY MUSIC",
20         "Code Breakers: TECH THRILLER CRIME",
21         "The Lost Kingdom: ADVENTURE ACTION FANTASY",
22         "Broken Silence: CRIME DRAMA SUSPENSE",
23         "Frozen Ashes: SURVIVAL DRAMA ACTION"
24     ]
25
26 # -----
27 # FUNCTION 2: Load Embedding Model + Generate Embeddings
28 # -----
29 def embed_movies(movies_list):
30     model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")
31     embeddings = model.embed_documents(movies_list)
32     return model, embeddings
33
34 # -----
35 # FUNCTION 3: Retrieve Top-K Relevant Movies
36 # -----
37 def retrieve_top_movies(query, model, movies_list, movie_embeddings, top_k
    ↪ =4):
38     query_emb = model.embed_query(query)
39
40     similarity_scores = {}
41     for i, emb in enumerate(movie_embeddings):
42         sim = cosine_similarity([query_emb], [emb])[0][0]
43         similarity_scores[movies_list[i]] = sim
44
45     sorted_results = sorted(
46         similarity_scores.items(), key=lambda x: x[1], reverse=True
47     )
48     return sorted_results[:top_k]
```

```
49
50 # -----
51 # FUNCTION 4: Build the RAG Prompt
52 # -----
53 def build_rag_prompt(query, retrieved_movies):
54     context_text = "\n".join([f"- {m[0]}" for m in retrieved_movies])
55
56     prompt = f"""
57 You are a movie recommendation expert.
58
59 USER QUERY:
60 {query}
61
62 RETRIEVED MOVIES (Context Provided):
63 {context_text}
64
65 TASK:
66 Using ONLY the above movies as your context, recommend movies to the user.
67 Clearly explain why each recommended movie matches the user's taste.
68 Do NOT mention similarity scores or embeddings.
69 """
70     return prompt
71
72 # -----
73 # FUNCTION 5: Invoke Ollama LLM
74 # -----
75 def generate_llm_response(prompt):
76     llm = OllamaLLM(model="qwen3:4b", temperature=0.7)
77
78     response = llm.invoke(prompt)
79     return response
80
81 # -----
82 # FUNCTION 6: Full RAG Pipeline Executor
83 # -----
84 def run_rag_pipeline():
85     # Load dataset
86     movies_list = load_movie_dataset()
87
88     # Embeddings
89     model, movie_embeddings = embed_movies(movies_list)
90
91     # User Input
92     query = input("Enter movie genre or mood: ")
93
94     # Retrieve
95     retrieved = retrieve_top_movies(query, model, movies_list,
```

```

    ↪ movie_embeddings)
96
97     # Build prompt
98     prompt = build_rag_prompt(query, retrieved)
99
100    # LLM response
101    answer = generate_llm_response(prompt)
102
103    print("\n===== RAG ANSWER =====")
104    print(answer)
105
106    # -----
107    # RUN PIPELINE
108    # -----
109    run_rag_pipeline()

```

74.1 Diff Explanation

The Ollama equivalent adapts the pipeline to use local models: HuggingFaceEmbeddings for retrieval and OllamaLLM for generation.

Key changes:

- **Removed:** `from FlagEmbedding import BGEM3FlagModel`
- **Removed:** `from langchain.chat_models import AzureChatOpenAI`
- **Added:** `from langchain_community.embeddings import HuggingFaceEmbeddings`
- **Added:** `from langchain_ollama import OllamaLLM`
- **Removed:** BGEM3FlagModel initialization
- **Added:** `model = HuggingFaceEmbeddings(model_name="BAAI/bge-m3")`
- **Removed:** `model.encode` for embeddings
- **Added:** `model.embed_documents` and `embed_query`
- **Removed:** AzureChatOpenAI with env vars
- **Added:** `llm = OllamaLLM(model="qwen3:4b", temperature=0.7)`
- **Removed:** `response.content`
- **Added:** `return response` (direct string)

These changes enable fully local execution, avoiding cloud dependencies for privacy and cost,

while maintaining the same modular RAG structure.

75 Hugging Face Equivalent Code

```

1  # =====
2  # RAG PIPELINE WITH EXTERNAL MOVIE DATASET -- MODULAR VERSION
3  # =====
4
5  from FlagEmbedding import BGEM3FlagModel
6  from sklearn.metrics.pairwise import cosine_similarity
7  from transformers import pipeline
8
9  # -----
10 # FUNCTION 1: Load Movie Dataset
11 # -----
12 def load_movie_dataset():
13     return [
14         "Shadow Hunters: ACTION FANTASY THRILLER",
15         "Eternal Bonds: DRAMA ROMANCE FAMILY",
16         "Neon Horizon: SCI-FI ACTION ADVENTURE",
17         "Whispers in the Dark: HORROR MYSTERY THRILLER",
18         "Crimson Crown: HISTORICAL WAR DRAMA",
19         "Starlight Dreams: ROMANCE COMEDY MUSIC",
20         "Code Breakers: TECH THRILLER CRIME",
21         "The Lost Kingdom: ADVENTURE ACTION FANTASY",
22         "Broken Silence: CRIME DRAMA SUSPENSE",
23         "Frozen Ashes: SURVIVAL DRAMA ACTION"
24     ]
25
26 # -----
27 # FUNCTION 2: Load Embedding Model + Generate Embeddings
28 # -----
29 def embed_movies(movies_list):
30     model = BGEM3FlagModel("BAAI/bge-m3", use_fp16=True)
31     embeddings = [
32         model.encode(movie, batch_size=12, max_length=8192)["dense_vecs"]
33         for movie in movies_list
34     ]
35     return model, embeddings
36
37 # -----
38 # FUNCTION 3: Retrieve Top-K Relevant Movies
39 # -----

```

```

40 def retrieve_top_movies(query, model, movies_list, movie_embeddings, top_k
    ↪ =4):
41     query_emb = model.encode(query, batch_size=12, max_length=8192)["
    ↪ dense_vecs"]
42
43     similarity_scores = {}
44     for i, emb in enumerate(movie_embeddings):
45         sim = cosine_similarity([query_emb], [emb])[0][0]
46         similarity_scores[movies_list[i]] = sim
47
48     sorted_results = sorted(
49         similarity_scores.items(), key=lambda x: x[1], reverse=True
50     )
51     return sorted_results[:top_k]
52
53 # -----
54 # FUNCTION 4: Build the RAG Prompt
55 # -----
56 def build_rag_prompt(query, retrieved_movies):
57     context_text = "\n".join([f"- {m[0]}" for m in retrieved_movies])
58
59     prompt = f"""
60 You are a movie recommendation expert.
61
62 USER QUERY:
63 {query}
64
65 RETRIEVED MOVIES (Context Provided):
66 {context_text}
67
68 TASK:
69 Using ONLY the above movies as your context, recommend movies to the user.
70 Clearly explain why each recommended movie matches the user's taste.
71 Do NOT mention similarity scores or embeddings.
72 """
73     return prompt
74
75 # -----
76 # FUNCTION 5: Invoke HF LLM
77 # -----
78 def generate_llm_response(prompt):
79     pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
80
81     result = pipe(prompt, max_new_tokens=200, temperature=0.7,
    ↪ num_return_sequences=1)
82     return result[0]['generated_text']
83

```



```

84 # -----
85 # FUNCTION 6: Full RAG Pipeline Executor
86 # -----
87 def run_rag_pipeline():
88     # Load dataset
89     movies_list = load_movie_dataset()
90
91     # Embeddings
92     model, movie_embeddings = embed_movies(movies_list)
93
94     # User Input
95     query = input("Enter movie genre or mood: ")
96
97     # Retrieve
98     retrieved = retrieve_top_movies(query, model, movies_list,
99     ↪ movie_embeddings)
100
101     # Build prompt
102     prompt = build_rag_prompt(query, retrieved)
103
104     # LLM response
105     answer = generate_llm_response(prompt)
106
107     print("\n===== RAG ANSWER =====")
108     print(answer)
109
110 # -----
111 # RUN PIPELINE
112 # -----
113 run_rag_pipeline()

```

75.1 Diff Explanation

The Hugging Face equivalent uses direct Hugging Face libraries: BGEM3FlagModel for embeddings and transformers pipeline for text generation.

Key changes:

- **Removed:** LangChain imports for LLM
- **Added:** from transformers import pipeline
- **Removed:** AzureChatOpenAI or OllamaLLM
- **Added:** pipe = pipeline("text-generation", model="Qwen/Qwen3-0.6B")
- **Removed:** llm.invoke(prompt)

- **Added:** `pipe(prompt, max_new_tokens=200, ...)`
- **Removed:** `response.content`
- **Added:** `result[0]['generated_text']`

These changes use native Hugging Face tools for both embedding and generation, providing more control over model parameters and avoiding LangChain abstractions for direct integration.